

CS3-7

รายงานทางความก้าวหน้าโครงการ ครั้งที่ 1

ระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการ
ประมวลผล ภาษาธรรมชาติแบบเฉพาะประเภท

Category-Specific News Classification and NLP Analytics
Pipeline

โดย

663380214-4 นายณราวิชญ์ ศิริพล

663380028-1 นายอัสดาวุฒิ เอื้อวงศ์

อาจารย์ที่ปรึกษา

อ.ดร.พบพร ด้านวิรุทย์

รายงานฉบับนี้เป็นส่วนหนึ่งของการศึกษา

วิชา CP354 772 โครงการคอมพิวเตอร์ 2

หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

ปีการศึกษา 2569

นราวิชญ์ ศิริพล และ อัสตาวุฒิ เอื้อวงศ์. 2569. ระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการประมวลผล ภาษาธรรมชาติแบบเฉพาะประเภท. โครงการคอมพิวเตอร์. ปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น.
อาจารย์ที่ปรึกษา: อ.ดร.พพพร ด้านวิรัช

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อศึกษา ออกแบบ และพัฒนาระบบ จำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการประมวลผล ภาษาธรรมชาติแบบเฉพาะประเภท (Category-Specific News Classification and NLP Analytics Pipeline) ที่ครอบคลุมการทำงานตั้งแต่การรวบรวมข่าวสารจากหลายสำนักข่าว การสกัดเนื้อหา การสรุปความ การจำแนกหมวดหมู่ การวิเคราะห์แนวโน้มความนิยม และการนำเสนอข้อมูลแบบเรียลไทม์ โดยสถาปัตยกรรมระบบได้รับการออกแบบตามหลักการ SOLID และ GRASP แบ่งการทำงานเป็นเลเยอร์อย่างเป็นระเบียบ ส่วนบริการเบื้องหลัง (Backend) พัฒนาด้วยภาษา Python และ FastAPI รองรับการดึงข้อมูลทั้งจาก RSS Feed และการสกรีนหน้าเว็บผ่าน Playwright Headless Browser พร้อมระบบแคชแบบหลายระดับ (Multi-tier Caching: หน่วยความจำและ HTTP 304) เพื่อลดภาระการเชื่อมต่อ การสรุปเนื้อหาและวิเคราะห์อารมณ์ของข่าว (Sentiment) ทำงานร่วมกับโมเดลภาษาขนาดใหญ่ (LLM) ผ่าน KGU Gen.AI API ขณะที่การจำแนกหมวดหมู่ข่าวภาษาไทย 7 หมวดหมู่หลักใช้แนวทางไฮบริดผสมกฎสำคัญเชิงบริบท (Rule-based Cues) ร่วมกับโมเดลการเรียนรู้ของเครื่อง (SVM/TF-IDF และ WangchanBERTa) ส่วนติดต่อผู้ใช้ (Frontend) พัฒนาด้วยรูปแบบ Single Page Application (SPA) ด้วย Vanilla JavaScript แบบ ES Modules และ Tailwind CSS สื่อสารกับเซิร์ฟเวอร์แบบเรียลไทม์ผ่าน WebSocket (Socket.IO) พร้อมฟังก์ชันหน้าต่างอ่านข่าวในระบบ (In-app Reader Modal) และระบบแนะนำข่าวสารตามความสนใจเฉพาะบุคคลโดยคำนึงถึงความเป็นส่วนตัวตามมาตรฐาน PDPA จากผลการประเมินการทำงานพบว่าระบบสามารถทำงานได้อย่างมีประสิทธิภาพ ตอบสนองคำขอรวดเร็ว และช่วยลดเวลาในการติดตามข่าวสารจากหลากหลายแหล่งได้อย่างมีประสิทธิภาพ

คำสำคัญ: การจำแนกข่าว, การวิเคราะห์ข้อความ, การประมวลผลภาษาธรรมชาติ, เว็บสแครปปิง, โมเดลภาษาขนาดใหญ่, อัลกอริทึมแนะนำข่าว

Narawit Siripol and Ausadawut Aueawong. 2026. **Category-Specific News Classification and NLP Analytics Pipeline**. Computer Project, Bachelor of Science, Program in Computer Science, College of Computing, Khon Kaen University.
Project Advisor: Dr. Pobporn Danvirutai

ABSTRACT

This project aims to study, design, and develop an automated Category-Specific News Classification and NLP Analytics Pipeline. The system covers an end-to-end workflow including multi-source news aggregation, content extraction, summarization, category classification, trend analytics, and real-time delivery. The architecture is engineered according to SOLID and GRASP software design principles with clearly decoupled layers. The backend is implemented in Python using FastAPI, supporting automated extraction from both standard RSS feeds and dynamic JavaScript-heavy websites via a Playwright headless browser instance. A multi-tier caching strategy (combining in-memory LRU caching and HTTP 304 conditional cache) is employed to minimize network overhead and redundant processing. For content summarization and sentiment analysis, the system integrates with Large Language Models (LLMs) via the KKU Gen.AI API to generate concise summaries, bullet points, and sentiment scores in the article's native language. Thai text classification into seven specialized categories is executed through a hybrid pipeline combining contextual keyword-based cues with machine learning models (TF-IDF with SVM and WangchanBERTa). The frontend is built as a responsive Single Page Application (SPA) using Vanilla JavaScript ES modules and Tailwind CSS, communicating bidirectionally with the server via WebSocket (Socket.IO). It provides an in-app modal reader, trending topic visualization, and privacy-compliant personalized recommendations based on aggregate engagement metrics under PDPA standards. The evaluation results demonstrate that the pipeline operates with high stability, low latency, and significantly reduces the effort required for readers to follow diverse news outlets.

Keywords: News classification, Text analytics, Natural language processing, Web scraping, Large language model, Recommendation algorithm

กิตติกรรมประกาศ

ข้าพเจ้าขอขอบพระคุณ อ.ดร.พพร ด้านวิรุฑย์ อาจารย์ที่ปรึกษาโครงการ ที่ได้กรุณาให้คำแนะนำ คำปรึกษา และข้อเสนอแนะอันเป็นประโยชน์ ตลอดระยะเวลาการดำเนินโครงการฉบับนี้ รวมทั้งสละเวลาในการตรวจแก้ไขและให้กำลังใจ จนทำให้โครงการนี้สำเร็จลุล่วงไปได้ด้วยดี

ขอขอบพระคุณ คณาจารย์ในสาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่นทุกท่านที่ได้ถ่ายทอดความรู้ทางวิชาการ ซึ่งเป็นพื้นฐานสำคัญในการศึกษาค้นคว้าและจัดทำโครงการนี้

ขอขอบพระคุณเพื่อนร่วมชั้นและผู้ที่เกี่ยวข้องทุกท่าน ที่ได้ให้ความช่วยเหลือ แนะนำ และแลกเปลี่ยนความคิดเห็น ซึ่งเป็นประโยชน์ต่อการดำเนินงานในหลายด้าน

ท้ายที่สุดนี้ ข้าพเจ้าขอขอบพระคุณบิดา มารดา และครอบครัว ที่ให้การสนับสนุน ส่งเสริม และเป็นกำลังใจ ตลอดระยะเวลาของการศึกษา

ผู้จัดทำ
นราวิชญ์ ศิริพล
อัสตาดุฒิ เอื้อวงศ์

สารบัญ

บทคัดย่อ	ก
ABSTRACT	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญภาพ	ช
สารบัญตาราง	ซ
1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตและข้อจำกัดของระบบ	2
1.3.1 ขอบเขต	2
1.3.2 ข้อจำกัดของโครงการ	3
1.4 ประโยชน์ที่คาดว่าจะได้รับ	4
2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	5
2.1 ทฤษฎีที่เกี่ยวข้อง	5
2.1.1 เว็บสแครปปิง (Web Scraping) และการดึงข้อมูลอัตโนมัติ	5
2.1.2 การสรุปข้อความด้วยโมเดลภาษาขนาดใหญ่ (LLM / Gemini)	5
2.1.3 การจำแนกข้อความภาษาไทยด้วย LinearSVC และ TF-IDF	6
2.1.4 การจำแนกเชิงลึกด้วยโมเดล Transformer (Wangchan-BERTa)	7
2.1.5 การจัดการแคชหลายระดับและระบบคุกกี้เพื่อความเป็นส่วนตัว	7
2.2 งานวิจัยที่เกี่ยวข้อง	8
2.2.1 งานสกัดเนื้อหาบทความจากเว็บ	8
2.2.2 งานสรุปข้อความอัตโนมัติ	8
2.2.3 งานจำแนกหมวดหมู่ข่าวภาษาไทย	8
2.2.4 สรุปผลการเปรียบเทียบงานวิจัยที่เกี่ยวข้อง	8
2.3 เหตุผลความจำเป็นในการเลือกเทคโนโลยีและเครื่องมือ (Tech Stack Rationales)	9
2.3.1 FastAPI	9
2.3.2 Pydantic	9

2.3.3	Playwright (Chromium Headless Browser)	9
2.3.4	PyThaiNLP	9
2.3.5	Socket.IO	9
2.3.6	ระบบจำลองเบราว์เซอร์แยกส่วนและเทคโนโลยี Obscura (Isolated Obscura CDP Engine)	10
2.3.7	ระบบจัดเก็บข้อมูลตามหลัก SOLID/GRASP (NewsRepositoryPort)	10
2.3.8	ระบบแคชหลายระดับ (Multi-tier Cache Strategy)	10
3	วิธีการดำเนินโครงการ	11
3.1	การฝึกสอน ประเมินผลโมเดล และแบบจำลองยูสเคส	11
3.1.1	ชุดข้อมูลและการเตรียมข้อมูล (Datasets & Data Preprocessing)	11
3.1.2	วิธีการฝึกสอนโมเดล (Model Training Methodology)	11
3.1.3	ผลการประเมินประสิทธิภาพและเมตริกวัดผล	12
3.1.4	เมทริกซ์ความสับสน (Confusion Matrix)	12
3.1.5	แบบจำลองยูสเคส (Use Case Diagram)	13
3.2	เครื่องมือที่ใช้ในการดำเนินงาน	14
3.2.1	ฮาร์ดแวร์	14
3.2.2	ซอฟต์แวร์	14
3.3	แผนการดำเนินงาน	16
4	การวิเคราะห์ ออกแบบและพัฒนาระบบ	17
4.1	การวิเคราะห์ระบบ	17
4.1.1	ผังแสดงทิศทางของกิจกรรม (Activity Flow)	17
4.1.2	แผนภาพลำดับขั้นตอนการทำงาน (Sequence Diagram)	19
4.1.3	รายการความต้องการของระบบ	19
4.2	การออกแบบระบบ	21
4.2.1	สถาปัตยกรรมของระบบและการแบ่งแยกโมดูล (System Architecture)	21
4.2.2	การออกแบบฐานข้อมูลและการจัดเก็บข้อมูล	22
4.3	การพัฒนาระบบและการทำงานของส่วนประกอบหลัก	23
4.3.1	กระบวนการดึงข่าวและการเชื่อมต่อ Playwright (Scraping Pipeline)	23
4.3.2	ระบบจำแนกหมวดหมู่และการตัดคำภาษาไทย (Thai NLP Classifier)	23
4.3.3	ระบบเปิดอ่านบทความในตัวเว็บ (In-App Reader) และการจำลองเบราว์เซอร์ด้วย Obscura	24
4.3.4	ระบบสรุปข่าวอัตโนมัติด้วยโมเดลภาษาขนาดใหญ่ (LLM Summarization Pipeline)	24
4.3.5	อัลกอริทึมแนะนำข่าวและจัดอันดับความนิยม (Trending & Hot News Algorithm)	25
4.3.6	การติดต่อสื่อสารแบบเรียลไทม์ผ่าน WebSocket	27

4.4	การออกแบบและมาตรการความมั่นคงปลอดภัยของระบบ (System Security Design)	27
4.4.1	ความมั่นคงปลอดภัยระดับเครือข่ายและเว็บเซิร์ฟเวอร์ (Network & Gateway Security)	27
4.4.2	การป้องกันการฉีดโค้ดอันตรายและการตรวจสอบข้อมูล (Data Sanitization & Injection Defense)	28
4.4.3	การคุ้มครองข้อมูลส่วนบุคคลและความปลอดภัยของคุกกี้ (Privacy & Cookie Security)	28
4.4.4	การจัดการความลับและการตั้งค่าระบบ (Secrets & Configuration Management)	29
4.5	การทดสอบระบบ	29
4.5.1	การทดสอบอัตโนมัติ (Automated Test Suite)	29
4.5.2	ผลการทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing)	29
5	สรุปผลการดำเนินงาน	31
5.1	สรุปผลการดำเนินโครงการ	31
5.2	ข้อจำกัดของระบบ	31
5.2.1	ข้อจำกัดด้านแหล่งข้อมูลภายนอก	31
5.2.2	ข้อจำกัดด้านทรัพยากรและการประมวลผล	32
5.3	ปัญหาอุปสรรคและแนวทางแก้ไข	32
5.4	ข้อเสนอแนะในการพัฒนาต่อไป	32
5.4.1	การปรับปรุงฟังก์ชันการทำงาน	32
5.4.2	แนวทางการวิจัยต่อยอด	32
	เอกสารอ้างอิง	33
	ภาคผนวก ก: คู่มือการติดตั้งและการใช้งานระบบ	35
	ภาคผนวก ข: ข้อกำหนด REST API และ WebSocket Events	37

สารบัญภาพ

ภาพที่ 1	แผนภาพแบบจำลองยูสเคสของระบบ (Use Case Diagram)	13
ภาพที่ 2	แผนภาพแสดงทิศทางการกิจกรรมของระบบ (Activity Diagram)	18
ภาพที่ 3	แผนภาพลำดับขั้นตอนการทำงานของระบบ (Sequence Diagram) . . .	19
ภาพที่ 4	แผนภาพสถาปัตยกรรมระบบและการแบ่งแยกโมดูล (System Architec- ture Diagram)	21

สารบัญตาราง

ตารางที่ 1	เปรียบเทียบงานวิจัยที่เกี่ยวข้อง	8
ตารางที่ 2	เปรียบเทียบประสิทธิภาพของโมเดลจำแนกหมวดหมู่ข่าว 9 หมวดหมู่ บนชุดข้อมูลทดสอบมาตรฐาน (Benchmark Test Set)	12
ตารางที่ 3	เมทริกซ์ความสับสน (Confusion Matrix) ของโมเดล 3-Tier Cascade บนชุดข้อมูลทดสอบมาตรฐาน (ThaiSum + Prachathai-67k)	12
ตารางที่ 4	รายการฮาร์ดแวร์ที่ใช้ในการพัฒนา	14
ตารางที่ 5	รายการซอฟต์แวร์และชุดเทคโนโลยีที่ใช้ในการพัฒนา (Exact Tech Stack Versions)	15
ตารางที่ 6	แผนการดำเนินงานและไทม์ไลน์ของโครงการ (Gantt Chart)	16
ตารางที่ 7	ความต้องการเชิงฟังก์ชันของระบบ (Functional Requirements)	20
ตารางที่ 8	การแบ่งชั้นสถาปัตยกรรมของระบบ (Backend Architecture Layers)	22
ตารางที่ 9	โครงสร้างข้อมูลบทความข่าว (NewsArticle Schema)	23
ตารางที่ 10	สรุปผลการเปรียบเทียบการคำนวณคะแนน Trending ระหว่างข่าวสด ใหม่และข่าวเก่า	27
ตารางที่ 11	ผลการทดสอบระบบตามกรณีทดสอบ (User Acceptance Testing)	30
ตารางที่ 12	สรุปผลการดำเนินงานเทียบกับวัตถุประสงค์	31
ตารางที่ 13	ปัญหา อุปสรรค และแนวทางการแก้ไข	32
ตารางที่ 14	รายละเอียด REST API Endpoints ของระบบ	37
ตารางที่ 15	รายละเอียด WebSocket Events สำหรับการสื่อสารแบบ Real-time	37

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบัน ข่าวสารเป็นข้อมูลสำคัญที่ช่วยให้ประชาชนรับรู้และติดตามเหตุการณ์ได้อย่างทันทั่วถึง โดยเฉพาะอย่างยิ่งในยุคที่ข้อมูลมีการเปลี่ยนแปลงตลอดเวลา อย่างไรก็ตาม ผู้อ่านที่ต้องการติดตามข่าวสารจากหลายแหล่งมักประสบปัญหาหลายประการ ได้แก่ ต้องเปิดเว็บไซต์หรือแอปพลิเคชันหลายแห่งเพื่อเปรียบเทียบข่าวเดียวกัน ทำให้เสียเวลาและเกิดความยุ่งยากในการติดตามสถานการณ์ที่เปลี่ยนแปลงอย่างรวดเร็ว นอกจากนี้ ข่าวแต่ละฉบับยังมีเนื้อหายาวและมีข้อมูลจำนวนมาก ทำให้ผู้อ่านต้องใช้เวลาในการอ่านเพื่อสกัดประเด็นสำคัญ รวมถึงยังต้องอ่านข่าวจากแหล่งภาษาอังกฤษเพื่อให้ได้มุมมองที่หลากหลาย อีกทั้งข่าวสารที่มีปริมาณมากและหลากหลายหมวดหมู่ยังทำให้การคัดกรองและวิเคราะห์ข่าวเป็นเรื่องท้าทาย

การนำเทคโนโลยีเว็บสแครปปิง (Web Scraping) และ RSS Feed มาใช้ในการดึงข้อมูลข่าวจากเว็บไซต์ข่าวอัตโนมัติ ตลอดจนการใช้เทคนิคการประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) ในการจำแนกหมวดหมู่ข่าวตามประเภท และการใช้โมเดลภาษาขนาดใหญ่ (Large Language Model: LLM) ในการสรุปเนื้อหา จะช่วยลดภาระของผู้อ่านในการเปิดหลายเว็บไซต์และอ่านบทความฉบับเต็ม อีกทั้งยังสามารถนำเสนอข้อมูลสำคัญที่คัดสรรและวิเคราะห์แล้วแบบเรียลไทม์ได้ในทีเดียว

จากปัญหาดังกล่าว โครงการนี้จึงมีแนวคิดในการพัฒนาระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการประมวลผลภาษาธรรมชาติแบบเฉพาะประเภท (Category-Specific NLP Analytics Pipeline) โดยระบบจะทำการดึงข่าวจากแหล่งข่าวเป็นระยะแบบอัตโนมัติ จำแนกหมวดหมู่ข่าวด้วยเทคนิค NLP และโมเดลแมชชีนเลิร์นนิง สรุปเนื้อหาด้วยโมเดลภาษาขนาดใหญ่ และแสดงผลข่าวพร้อมการวิเคราะห์บนเว็บแอปพลิเคชันแบบเรียลไทม์ผ่าน WebSocket เพื่อให้ผู้ใช้สามารถติดตามข่าวสารได้อย่างสะดวก รวดเร็ว และเหมาะสมกับความสนใจของแต่ละบุคคล

1.2 วัตถุประสงค์

- 1) เพื่อศึกษาและวิเคราะห์เทคนิคการดึงข้อมูลข่าวจากเว็บไซต์ข่าวด้วยวิธีเว็บสแครปปิง และการอ่าน RSS Feed รวมถึงเทคนิคการประมวลผลภาษาธรรมชาติสำหรับการจำแนกหมวดหมู่ข่าวตามประเภท
- 2) เพื่อออกแบบและพัฒนาระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการประมวลผลภาษาธรรมชาติแบบเฉพาะประเภท (Category-Specific NLP Analytics Pipeline) ครอบคลุมการดึงข่าว การจำแนกหมวดหมู่ การสรุปเนื้อหาด้วยโมเดลภาษาขนาดใหญ่ อัลกอริทึมแนะนำข่าว และการแสดงผลแบบเรียลไทม์
- 3) เพื่อทดสอบและประเมินประสิทธิภาพของระบบในด้านความถูกต้องของการจำแนกหมวดหมู่ข่าว ความถูกต้องของการสรุปข่าว ความแม่นยำของอัลกอริทึมแนะนำข่าว

และความสามารถในการแสดงผลข่าวแบบเรียลไทม์

1.3 ขอบเขตและข้อจำกัดของระบบ

1.3.1 ขอบเขต

ระบบเป็นเว็บแอปพลิเคชันแบบเต็มรูปแบบ (Full-stack) ที่มีขอบเขตการทำงาน ดังนี้

- 1) **การรวบรวมข่าวอัตโนมัติ:** ระบบสามารถดึงข่าวจากแหล่งข่าวออนไลน์หลายแห่งโดยใช้เทคนิคเว็บสแครปปิง การอ่าน RSS Feed และการจำลองเบราว์เซอร์ด้วย Playwright สำหรับเว็บไซต์ที่ใช้ JavaScript
- 2) **การสรุปข่าวและวิเคราะห์อารมณ์:** ระบบส่งเนื้อหาข่าวในรูปแบบ Markdown ให้โมเดลภาษาขนาดใหญ่ผ่าน API ของมหาวิทยาลัยขอนแก่น (KKU Gen.AI) เพื่อสร้างบทสรุป ประเด็นสำคัญ อารมณ์ของข่าว (Sentiment) และคำสำคัญ โดยสรุปเป็นภาษาเดียวกับต้นฉบับข่าว
- 3) **การจำแนกหมวดหมู่ข่าว 9 หมวดหมู่หลัก:** ระบบจำแนกข่าวออกเป็น 9 หมวดหมู่หลัก ได้แก่ การเมือง (politics), เศรษฐกิจ (economy), เทคโนโลยี (tech), สุขภาพ (health), สิ่งแวดล้อม (environment), กีฬา (sports), บันเทิง (entertainment), สังคม (society) และต่างประเทศ (world) โดยใช้การตัดคำภาษาไทยด้วย PyThaiNLP ร่วมกับโมเดลการเรียนรู้ของเครื่องแบบหลายชั้น (3-Tier Cascade Classifier: Rule Cues, Calibrated LinearSVC และ WangchanBERTa)
- 4) **การจัดเก็บและวงจรชีวิตของข้อมูล (Data Lifecycle & Retention):** ข้อมูลข่าวและผลการวิเคราะห์ทั้งหมดจะถูกจัดเก็บรวบรวมอย่างต่อเนื่องนับตั้งแต่วันที่ Instance Server เริ่มทำงาน (Instance Startup Date) โดยจัดเก็บไว้ภายใน Docker Container และ Local Storage บนเครื่องแม่ข่าย AWS EC2
- 5) **ขอบเขตความเป็นส่วนตัวและการไหลของข้อมูลผู้ใช้ (User Privacy & Data Flow):**
 - **ข้อมูลที่ถูกประมวลผล:** ระบบบันทึกเฉพาะรหัสเซสชันแบบไม่ระบุตัวตน (Anonymous Cookie ID) และสถิติพฤติกรรมการอ่านข่าวแบบภาพรวม (Non-PII Aggregate Engagement: การคลิกเปิดอ่าน, การขยายอ่านบทสรุป, การบุ๊กมาร์ก) โดยเก็บค่ากำหนดไว้ใน Local Cookies บนเว็บเบราว์เซอร์ของผู้ใช้ และส่งสถิติรวมมายังเซิร์ฟเวอร์เพื่อใช้คำนวณคะแนนความนิยม (Trending Score)
 - **ข้อมูลที่ไม่ถูกจัดเก็บและไม่ถูกส่งต่อ:** ระบบไม่มีการจัดเก็บข้อมูลระบุตัวตนส่วนบุคคล (No Personally Identifiable Information: PII เช่น ชื่อ-นามสกุล, อีเมล, หมายเลขโทรศัพท์ หรือ IP Address), ไม่มีการส่งข้อมูลพฤติกรรมของผู้ใช้ไปยังโมเดลภาษาขนาดใหญ่ภายนอก (KKU Gen.AI จะได้รับเฉพาะเนื้อหาข่าวเท่านั้น) และไม่มีการส่งต่อหรือขายข้อมูลให้แก่บุคคลภายนอกหรือบริการวิเคราะห์ภายนอกใด ๆ
- 6) **การแสดงผลแบบเรียลไทม์:** ระบบแสดงรายการข่าวบนเว็บแอปพลิเคชันพร้อมสรุปเนื้อหา ภาพประกอบ และหมวดหมู่ และแจ้งเตือนข่าวใหม่ทันทีผ่าน WebSocket (Socket.IO) โดยไม่ต้องรีเฟรชหน้าเว็บ

- 7) **การอ่านข่าวในตัวเว็บ (In-app Reader Modal):** ระบบ มีหน้า ต่าง แสดง ผล บทความข่าวและบทวิเคราะห์ในตัวเว็บแอปพลิเคชัน เพื่อให้ผู้ใช้ อ่านข่าวได้อย่าง ต่อเนื่องโดยไม่ต้องเปิดแท็บใหม่
- 8) **การจัดเก็บข้อมูลและสถาปัตยกรรมแบบแยกชั้น:** ระบบจัดเก็บข้อมูลข่าวและสถิติ การใช้งานในรูปแบบ JSON บนเครื่องเซิร์ฟเวอร์ ออกแบบตามหลัก SOLID/GRASP ด้วย Port/Adapter Pattern
- 9) **การ ตั้ง ค่า และ ความ ยืดหยุ่น:** ระบบ รองรับ การ เพิ่ม แหล่ง ข่าว ใหม่ ได้ โดย ไม่ ต้อง แก้ไข โครงสร้าง หลัก ผ่าน การ ลง ทะเบียน Source ด้วย Decorator (@register_source)
- 10) **ระบบแคชหลายระดับ (Multi-tier Cache):** ระบบมีกลไกจัดเก็บข้อมูลข่าวและ เนื้อหาในหน่วยความจำ (AsyncInMemoryCache) ร่วมกับแคชแบบมีเงื่อนไข (HTTP 304 Conditional Cache) เพื่อลดการดึงข้อมูลซ้ำและเพิ่มความรวดเร็วในการแสดงผล
- 11) **การ รองรับ Progressive Web App (PWA):** รองรับ การ ทำงาน แบบ เว็บ แอปพลิเคชัน ก้าวหน้า พร้อม Service Worker สำหรับการ แคช หน้า เว็บ เพื่อ ความรวดเร็ว

1.3.2 ข้อจำกัดของโครงการ

- 1) **ข้อจำกัดด้านทรัพยากรฮาร์ดแวร์และการติดตั้งใช้งาน:** ระบบถูกออกแบบและปรับ แต่งให้รองรับภาระงานได้ตามขีดความสามารถของเครื่องแม่ข่าย AWS EC2 อินสแตนซ์ ประเภท `c7i-flex.large` (2 vCPU, หน่วยความจำหลัก RAM ขนาด 4 GiB และ Swap Buffer ขนาด 4 GB) ซึ่งอาจมีข้อจำกัดในการรองรับผู้ใช้งานพร้อมกัน จำนวนมหาศาล (Concurrent Users) หรือการรันเบราร์เซอร์ Playwright จำนวนหลายอินสแตนซ์พร้อมกัน
- 2) **ข้อจำกัดของชุดข้อมูลฝึกสอนโมเดล (Training Dataset Constraints):** โมเดล การเรียนรู้ของเครื่องสำหรับการจำแนกหมวดหมู่ข่าวได้รับการฝึกสอนและ ประเมิน ผลบนชุดข้อมูลจำกัดจากคลังข้อมูล ThaiSum และ Prachathai-67k ซึ่งอาจมีความ ครอบคลุมของคำศัพท์เฉพาะกลุ่มหรือโครงสร้างภาษาตามลักษณะของแหล่งข่าวดัง กล่าว
- 3) ระบบรองรับการสรุปและแสดงผลข่าวในภาษาไทยและภาษาอังกฤษเท่านั้น ตามภาษา เดิมของเนื้อหาข่าวแต่ละฉบับ
- 4) โครงสร้าง HTML หรือ RSS Feed ของเว็บไซต์แหล่งข่าวอาจเปลี่ยนแปลงได้ ทำให้ ระบบต้องได้รับการปรับปรุงให้สอดคล้อง
- 5) การสรุปข่าวต้องอาศัยการเชื่อมต่ออินเทอร์เน็ตและ API ของโมเดลภาษาขนาดใหญ่ (KKU Gen.AI) หากบริการขัดข้องหรือหมดโควตาการใช้งาน ระบบจะทำงานในโหมด ข้อความสรุปตั้งต้นแทน
- 6) รอบแรกของการดึงข่าวอาจใช้เวลา 2–5 นาที เนื่องจากต้องดึงเนื้อหาบทความของ แต่ละข่าวเพื่อใช้ในการสรุป

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1) ได้ระบบเว็บแอปพลิเคชันจำแนกและวิเคราะห์ข่าวสารอัตโนมัติแบบเรียลไทม์ ที่ช่วยให้ผู้ใช้ติดตามข่าวสารจากหลายแหล่งได้ในที่เดียวอย่างสะดวก รวดเร็ว และเหมาะสมกับความสนใจของแต่ละบุคคล
- 2) ได้แนวทางในการประยุกต์ใช้เทคนิคเว็บสแครปปิง การอ่าน RSS Feed โมเดลภาษาขนาดใหญ่ อัลกอริทึมการกำหนดคูกี้เพื่อการปรับแต่งเฉพาะบุคคล และอัลกอริทึมแนะนำข่าว เพื่อสร้างระบบวิเคราะห์ข้อมูลข่าวอัตโนมัติ ซึ่งสามารถนำไปพัฒนาต่อยอดกับข้อมูลประเภทอื่น
- 3) ได้องค์ความรู้ด้านการประมวลผลภาษาธรรมชาติ การจำแนกข้อความด้วยกฎคำสำคัญ ร่วมกับโมเดลแมชชีนเลิร์นนิง เทคนิคการสรุปข้อความด้วยโมเดลภาษาขนาดใหญ่ และการพัฒนาระบบเว็บแบบเรียลไทม์

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีที่เกี่ยวข้อง

2.1.1 เว็บสแครปปิง (Web Scraping) และการดึงข้อมูลอัตโนมัติ

เว็บสแครปปิงคือกระบวนการดึงข้อมูลจากเว็บไซต์โดยอัตโนมัติ แทนการคัดลอกข้อมูลด้วยมือ โดยทั่วไปจะส่งคำขอ HTTP ไปยังเว็บเซิร์ฟเวอร์แล้ววิเคราะห์โครงสร้าง HTML ของหน้าเว็บเพื่อสกัดข้อมูลที่ต้องการ [1] เครื่องมือที่นิยมใช้ ได้แก่ ไลบรารี BeautifulSoup สำหรับวิเคราะห์และค้นหาองค์ประกอบในเอกสาร HTML ด้วย CSS Selector และไลบรารี httpx สำหรับส่งคำขอ HTTP แบบอะซิงโครนัส อย่างไรก็ตาม เว็บไซต์บางแห่งแสดงผลเนื้อหาผ่าน JavaScript ต้องใช้เครื่องมือจำลองเบราว์เซอร์ เช่น Playwright เพื่อให้ได้ข้อมูลที่แสดงผลสมบูรณ์

นอกจากนี้ เว็บไซต์หลายแห่งยังเผยแพร่ข่าวผ่าน RSS Feed ซึ่งเป็นรูปแบบ XML มาตรฐานที่ใช้แจกจ่ายเนื้อหาที่อัปเดตเป็นประจำ ทำให้สามารถดึงหัวข้อข่าว ลิงก์ และบทสรุปสั้น ๆ ได้โดยไม่ต้องวิเคราะห์โครงสร้าง HTML โดยตรง [2] ในระบบนี้ใช้ทั้งสองเทคนิคร่วมกันตามความเหมาะสมของแต่ละสำนักข่าว

2.1.2 การสรุปข้อความด้วยโมเดลภาษาขนาดใหญ่ (LLM / Gemini)

การสรุปข้อความ (Text Summarization) แบ่งเป็น 2 แนวทางหลัก ได้แก่:

- 1) **การสรุปแบบดึงประโยค (Extractive Summarization):** ใช้อัลกอริทึมทางสถิติหรือกราฟ เช่น TextRank ในการคัดเลือกประโยคที่มีอยู่เดิมในบทความ ข้อจำกัดคือมักได้ข้อความที่ขาดความต่อเนื่องและมีความเยิ่นเย้อ
- 2) **การสรุปแบบสร้างข้อความใหม่ (Abstractive Summarization):** ทำความเข้าใจบริบทของเนื้อหาทั้งหมดแล้วสังเคราะห์เป็นข้อความสรุปชุดใหม่ด้วยภาษาที่เป็นธรรมชาติ [3]

(1) เหตุผลที่เลือกใช้ Large Language Model (Gemini ผ่าน KGU Gen.AI)

โครงการนี้เลือกใช้โมเดลภาษาขนาดใหญ่ตระกูล Gemini ผ่านบริการ KGU Gen.AI API [4] ด้วยเหตุผลสำคัญดังนี้:

- **ความสามารถด้าน Abstractive และความเข้าใจภาษาไทย:** มีความสามารถในการจับประเด็นสำคัญของบทความข่าวขนาดยาว เรียบเรียงเป็นภาษาไทยที่กระชับ สละสลวย และถูกต้องตามหลักไวยากรณ์
- **การสร้างผลลัพธ์แบบมีโครงสร้าง (Structured JSON Output):** สามารถควบคุมให้โมเดลตอบกลับในรูปแบบ JSON ตาม Schema ที่กำหนดได้อย่างเคร่งครัด ทำให้ได้ทั้งบทสรุปกระชับ 2-3 ประโยค, ประเด็นสำคัญ (Bullet Points), การวิเคราะห์อารมณ์

ของข่าว (Sentiment: Positive, Neutral, Negative) และคำสำคัญ (Keywords) ในการเรียกใช้เพียงรอบเดียว

- **รองรับเนื้อหาหลายภาษา (Multilingual Support):** สามารถสรุปเนื้อหาเป็นภาษาเดียวกับต้นฉบับข่าวได้อัตโนมัติ (ข่าวไทยสรุปเป็นไทย ข่าวต่างประเทศสรุปเป็นอังกฤษ)
- **ประสิทธิภาพและความคุ้มค่า (Efficiency & Zero-GPU Constraint):** การเรียกใช้งานผ่าน API ช่วยให้เซิร์ฟเวอร์ของระบบไม่จำเป็นต้องติดตั้งการ์ดจอประมวลผลขั้นสูง (High-end GPU) ช่วยลดภาระทรัพยากรฮาร์ดแวร์ของระบบได้อย่างมหาศาล

2.1.3 การจำแนกข้อความภาษาไทยด้วย LinearSVC และ TF-IDF

การจำแนกข้อความ (Text Classification) มีขั้นตอนสำคัญในการแปลงข้อความตัวอักษรให้อยู่ในรูปเวกเตอร์ตัวเลข (Feature Extraction) และการใช้โมเดลการเรียนรู้ของเครื่องในการแบ่งแยกหมวดหมู่

(1) ทฤษฎี TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF เป็นค่าน้ำหนักเชิงสถิติที่ใช้วัดความสำคัญของคำ t ในเอกสาร d เทียบกับคลังเอกสารทั้งหมด D :

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (1)$$

โดยที่ $\text{TF}(t, d)$ คือความถี่ของคำ t ในเอกสาร d และ $\text{IDF}(t, D) = \log\left(\frac{1+|D|}{1+|\{d \in D: t \in d\}|}\right) + 1$ ช่วยลดน้ำหนักของคำที่ปรากฏบ่อยทั่วไปในทุกข่าว และเพิ่มน้ำหนักให้คำเฉพาะกลุ่มหมวดหมู่

(2) ทฤษฎี Support Vector Machine (LinearSVC)

Linear Support Vector Classifier (LinearSVC) เป็นโมเดลที่มุ่งหาเส้นแบ่งระนาบหลายมิติ (Hyperplane) ที่มีระยะห่าง (Margin) จากจุดข้อมูลของแต่ละคลาสมากที่สุด:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2)$$

(3) เหตุผลที่เลือกใช้ LinearSVC ร่วมกับ TF-IDF

- **ความเหมาะสมกับข้อมูลมิติสูงแบบเบาบาง (High-dimensional Sparse Data):** เวกเตอร์ข้อความที่ได้จาก TF-IDF มักมีมิติหลายพันถึงหลายหมื่นมิติ ซึ่ง Linear Kernel ของ SVM มีคุณสมบัติเด่นในการแบ่งกลุ่มในมิติสูงได้อย่างแม่นยำ และมี L_2 Regularization ที่ช่วยป้องกันปัญหา Overfitting ได้ดีกว่า Decision Tree หรือ Naive Bayes
- **ความเร็วในการทำนายที่สูงมาก (Sub-millisecond Inference):** การคำนวณแบบ Linear ทำได้เร็วมาก (ใช้เวลาต่ำกว่า 2-5 มิลลิวินาทีต่อบทความ) ทำให้ระบบสามารถประมวลผลจำแนกข่าวจำนวนมากใน Background Task ได้ทันทีโดยไม่หน่วง Event Loop
- **ประหยัดทรัพยากรหน่วยความจำและ CPU:** โมเดลที่เทรนเสร็จแล้วมีขนาดไฟล์เพียง

ไม่กี่เมกะไบต์ (Megabytes) และใช้หน่วยความจำขณะรันน้อยมาก

2.1.4 การ จำแนก เชิง ลึก ด้วย โมเดล Transformer (WangchanBERTa)

WangchanBERTa [5] เป็นโมเดลภาษาขนาดใหญ่ที่ใช้สถาปัตยกรรม RoBERTa (Robustly Optimized BERT Approach) ผ่านการฝึกฝนล่วงหน้า (Pre-trained) บนคลังข้อมูลภาษาไทยขนาดใหญ่กว่า 78 GB

(1) เหตุผลที่ประยุกต์ใช้ WangchanBERTa ในระบบ

- **การเข้าใจบริบทสองทิศทาง (Bidirectional Contextual Attention):** กลไก Multi-head Self-Attention ช่วยให้โมเดลเข้าใจคำพ้องรูป คำกำกวม หรือคำสแลงในหัวข้อข่าวที่โมเดล Bag-of-Words ทั่วไปไม่สามารถจับความสัมพันธ์ระหว่างคำที่อยู่ห่างกันได้
- **การเป็นมาตรฐานอ้างอิงเพื่อการวิจัย (Deep Learning Benchmark):** ระบบนำ WangchanBERTa มาใช้เป็นโมเดลทางเลือกสำหรับการวิจัยและเปรียบเทียบประสิทธิภาพกับโมเดล Linear เพื่อประเมินความคุ้มค่าระหว่างความแม่นยำกับระยะเวลาการประมวลผล

2.1.5 การจัดการแคชหลายระดับและระบบคุกกี้เพื่อความเป็นส่วนตัว

(1) ระบบแคชหลายระดับ (Multi-tier Cache Architecture)

เพื่อตอบสนองต่อผู้ใช้ได้อย่างรวดเร็วและประหยัดการเชื่อมต่อภายนอก ระบบออกแบบระบบแคชสองระดับ:

- 1) **แคชระดับหน่วยความจำ (In-Memory LRU Cache):** AsyncInMemoryCache จัดเก็บผลลัพธ์การสกัดข่าวและผลสรุปจาก LLM ใน RAM ช่วยให้คำขอข่าวที่เปิดซ้ำตอบสนองได้ในเวลาต่ำกว่า 5 ms และป้องกันการยิง API ซ้ำซ้อน
- 2) **แคชระดับโพรโทคอล (HTTP 304 Conditional Cache):** ใช้กลไก ETag และ Header If-Modified-Since ตรวจสอบว่าหน้าเว็บข่าวต้นทางมีการเปลี่ยนแปลงหรือไม่ หากไม่มีการเปลี่ยนแปลง เซิร์ฟเวอร์จะคืนค่า HTTP 304 Not Modified ทำให้ไม่ต้องดาวน์โหลดเนื้อหาหน้าเว็บซ้ำ ช่วยลดทราฟฟิกเครือข่ายและป้องกันการถูกระงับการเข้าถึง (IP Ban)

(2) ระบบคุกกี้และความเป็นส่วนตัว (Privacy-Preserving Cookies & PDPA)

ระบบใช้คุกกี้ในการจัดเก็บค่ากำหนดเฉพาะบุคคล (Personalize Cookies) และบันทึกคะแนนปฏิสัมพันธ์ของผู้ใช้ (Algorithm Cookies) สำหรับจัดอันดับความนิยมของข่าว (Trending Algorithm) โดยคำนึงถึงพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล (PDPA) ด้วยการไม่จัดเก็บข้อมูลระบุตัวตนส่วนบุคคล (Non-PII Aggregate Metrics) และมีแถบแจ้งเตือนขอความยินยอมการใช้คุกกี้ (Cookie Consent Banner)

2.2 งานวิจัยที่เกี่ยวข้อง

ในส่วนนี้จะนำเสนองานวิจัยหรือระบบที่มีลักษณะ คล้ายคลึงกับ โครงการนี้ เพื่อเป็นแนวทางในการพัฒนา โดยพิจารณาครอบคลุมในด้านการสกัดเนื้อหาจากเว็บ การสรุปข้อความ และการจำแนกหมวดหมู่ข่าว

2.2.1 งานสกัดเนื้อหาบทความจากเว็บ

Barbaresi [1] ได้พัฒนาไลบรารี Trafilatura สำหรับสกัดข้อความหลักออกจากหน้าเว็บ โดยอัตโนมัติ โดยไม่ขึ้นกับเว็บไซต์ใดเว็บไซต์หนึ่ง ไลบรารีนี้รองรับการจัดรูปแบบ HTML ที่หลากหลายและคืนข้อความในรูปแบบ Markdown ซึ่งเป็นประโยชน์โดยตรงต่อการนำเนื้อหาไปใช้กับโมเดลภาษาขนาดใหญ่ อย่างไรก็ตาม วิธีนี้มีข้อจำกัดเมื่อเนื้อหาถูกเรนเดอร์ด้วย JavaScript บางกรณี ทำให้ระบบนี้ต้องเสริมด้วยการจำลองเบราว์เซอร์ด้วย Playwright สำหรับเว็บไซต์กลุ่มดังกล่าว

2.2.2 งานสรุปข้อความอัตโนมัติ

Allahyari และคณะ [3] ได้สำรวจเทคนิคการสรุปข้อความอย่างเป็นระบบทั้งแบบ Extract และ Abstractive และชี้ให้เห็นว่าการสรุปแบบ Abstractive ที่ใช้การเรียนรู้เชิงลึกให้ผลลัพธ์ที่เป็นธรรมชาติกว่าอย่างมีนัยสำคัญ แต่ต้องใช้ทรัพยากรในการคำนวณสูง ด้วยความก้าวหน้าของโมเดลภาษาขนาดใหญ่ [4] การสรุปแบบ Abstractive จึงเข้าถึงได้ง่ายผ่าน API โดยไม่ต้องฝึกโมเดลเอง โครงการนี้จึงเลือกใช้แนวทางดังกล่าวเพื่อสรุปข่าวเป็นภาษาไทยด้วยต้นฉบับโดยไม่ต้องใช้ทรัพยากรฝึกโมเดลของตนเอง

2.2.3 งานจำแนกหมวดหมู่ข่าวภาษาไทย

งานวิจัยด้านการจำแนกข้อความภาษาไทยมักประสบปัญหาการเว้นวรรคคำ จึงนิยมใช้เครื่องมือตัดคำภาษาไทย เช่น PyThaiNLP [6] ร่วมกับโมเดลแมชชีนเลิร์นนิง เช่น SVM กับคุณลักษณะ TF-IDF ซึ่งเป็นแนวทางที่มีประสิทธิภาพและใช้ทรัพยากรต่ำ โครงการนี้ประยุกต์ใช้แนวทางไฮบริด โดยใช้กฎคำสำคัญที่มีน้ำหนักและลำดับความสำคัญของ URL สำหรับการจำแนกแบบรวดเร็ว และใช้โมเดล SVM เป็นตัวตรวจสอบบริบทสำหรับหมวดหมู่ที่กฎคำสำคัญมีความแม่นยำไม่เพียงพอ

2.2.4 สรุปผลการเปรียบเทียบงานวิจัยที่เกี่ยวข้อง

จากการทบทวนวรรณกรรมข้างต้น สามารถสรุปผลการเปรียบเทียบคุณสมบัติได้ดังตารางที่ 1

ตารางที่ 1 เปรียบเทียบงานวิจัยที่เกี่ยวข้อง

งานวิจัย	สกัดเนื้อหา	สรุป LLM	จำแนกหมวด	Real-time	แคช/คุกกี้	ระบบแนะนำ
Trafilatura [1]	✓	–	–	–	–	–
งานสรุปข้อความ [3]	–	✓	–	–	–	–
งานจำแนกข้อความภาษาไทย [6]	–	–	✓	–	–	–
โครงการนี้	✓	✓	✓	✓	✓	✓

จากตารางที่ 1 พบว่ายังไม่มีงานใดที่ผสมผสานการสกัดเนื้อหา การสรุปด้วยโมเดลภาษา

ขนาดใหญ่ การจำแนกหมวดหมู่ การแสดงผลแบบเรียลไทม์ การจัดการแคชและคุกกี้ และอัลกอริทึมแนะนำข่าวในระบบเดียวครบทุกด้าน โครงการนี้จึงมุ่งพัฒนาเว็บแอปพลิเคชันที่รวบรวมคุณสมบัติทั้งหมดไว้ในระบบเดียวอย่างครบถ้วน

2.3 เหตุผลความจำเป็นในการเลือกเทคโนโลยีและเครื่องมือ (Tech Stack Rationales)

การเลือกชุดเทคโนโลยี (Tech Stack) ในโครงการนี้คำนึงถึงประสิทธิภาพการทำงาน ความเข้ากันได้ของระบบ และมาตรฐานทางวิศวกรรมซอฟต์แวร์ โดยมีเหตุผลในการเลือกใช้แต่ละเทคโนโลยี ดังนี้

2.3.1 FastAPI

FastAPI [7] ถูกเลือกเป็นเฟรมเวิร์คหลักของส่วน Backend เนื่องจากทำงานบนสถาปัตยกรรมแบบ Asynchronous (ASGI บน Starlette) ซึ่งเหมาะสมอย่างยิ่งสำหรับงานที่เป็น Non-blocking I/O เช่น การรัน Background Scraper พร้อมกันหลายแหล่งข่าว การเชื่อมต่อ WebSocket แบบเรียลไทม์ และการเรียก API ภายนอก นอกจากนี้ยังมีความเร็วในการประมวลผลเทียบเท่าภาษา Go หรือ Node.js และสร้างเอกสาร OpenAPI Specification อัตโนมัติ

2.3.2 Pydantic

Pydantic [8] ถูกนำมาใช้ในการควบคุมและตรวจสอบชนิดข้อมูล (Data Validation & Serialization) ที่ระดับ Runtime โดยใช้ Type Hints ของภาษา Python การใช้ Pydantic ช่วยรับประกันว่าข้อมูลข่าวที่สกัดได้จากเว็บและข้อมูลผลลัพธ์จาก LLM จะถูกต้องตรงตาม Schema ที่กำหนดไว้เสมอ ป้องกันปัญหาข้อมูลสูญหายหรือผิดรูปแบบก่อนจัดเก็บลงฐานข้อมูล

2.3.3 Playwright (Chromium Headless Browser)

Playwright [9] ถูกเลือกใช้สำหรับการดึงข้อมูลหน้าเว็บที่เรนเดอร์ด้วย JavaScript (Single Page Application เช่น React/Vue) และเว็บไซต์ที่มีกลไกป้องกันบอต เช่น Cloudflare Challenge โดย Playwright สามารถจำลองเบราว์เซอร์จริงได้อย่างสมบูรณ์ จัดการ Cookies และ Browser Context แยกกันอย่างอิสระ มีความเสถียรและทำงานได้เร็วกว่า Selenium

2.3.4 PyThaiNLP

PyThaiNLP [6] เป็นเครื่องมือประมวลผลภาษาธรรมชาติภาษาไทยมาตรฐาน โดยระบบเลือกใช้อัลกอริทึมตัดคำ newmm (Dictionary-based Maximal Matching ผสานกับ Thai Character Clusters: TCC) ร่วมกับชุดคำหยุดภาษาไทย (Thai Stop words) เพื่อเตรียมข้อความก่อนนำไปสร้างเวกเตอร์ TF-IDF

2.3.5 Socket.IO

Socket.IO [10] ถูกเลือกสำหรับการสื่อสารแบบ Real-time ระหว่างเซิร์ฟเวอร์กับเว็บเบราว์เซอร์ เนื่องจากมีฟีเจอร์การเชื่อมต่อใหม่โดยอัตโนมัติ (Auto-reconnection), ระบบ Fallback เป็น HTTP Long-polling กรณีที่เครือข่ายไม่อนุญาตให้ใช้ WebSocket และระบบจัดการ Event-driven architecture ที่เรียบง่ายและเสถียร

2.3.6 ระบบจำลองเบราว์เซอร์แยกส่วนและเทคโนโลยี Obscura (Isolated Obscura CDP Engine)

Obscura (h4ckf0r0day/obscura) ถูกนำมาใช้เป็นเอนจินจำลองเบราว์เซอร์แบบแยกส่วน (Isolated Headless Browser Container) โดยสื่อสารผ่านโปรโตคอล Chrome DevTools Protocol (CDP) ผ่านการเชื่อมต่อ WebSocket (`ws://obscura:9222`) ซึ่งมีประโยชน์เชิงวิศวกรรมที่สำคัญ 3 ประการ:

- 1) **การแยกทรัพยากร (Resource Isolation):** การประมวลผลเรนเดอร์หน้าเว็บข่าวที่ซับซ้อนและมี JavaScript ปริมาณมากจะถูกแยกไปทำงานใน Container ของ Obscura โดยไม่รบกวนหน่วยความจำและ CPU ของ FastAPI Backend ช่วยป้องกันปัญหาหน่วยความจำล้น (Out-of-Memory: OOM) ภายใต้ทรัพยากรที่จำกัดของ AWS EC2 `c7i-flex.large` (4 GiB Memory)
- 2) **การพรางตัวหลบเลี่ยงการตรวจจับบอต (Stealth Bypass & Anti-Bot Evasion):** Obscura ถูกปรับแต่งโครงสร้าง Browser Fingerprinting และ Canvas/WebGL เพื่อลดโอกาสการถูกบล็อกคำขอจากเว็บไซต์ข่าวที่มีระบบตรวจจับบอตขั้นสูง
- 3) **กลไกสำรองการทำงานที่ไร้รอยต่อ (Fault-Tolerant Graceful Fallback):** คลาส `BrowserService` ถูกออกแบบให้เชื่อมต่อไปยัง Obscura CDP ก่อนเสมอ หากคอนเทนเนอร์ภายนอกไม่พร้อมใช้งาน ระบบจะสลับไปรัน Local Playwright Chromium Engine ภายในตัวโดยอัตโนมัติ ทำให้ระบบมีความต่อเนื่องในการให้บริการ (High Availability)

2.3.7 ระบบจัดเก็บข้อมูลตามหลัก SOLID/GRASP (NewsRepositoryPort)

การออกแบบระบบจัดเก็บข้อมูลใช้ Port and Adapter Pattern ผ่านโปรโตคอล `NewsRepositoryPort` ทำให้โค้ดส่วนบริการไม่ผูกติดกับเทคโนโลยีฐานข้อมูลใดฐานข้อมูลหนึ่ง โดยระบบเริ่มต้นพัฒนาด้วย `FileNewsRepository` ที่จัดเก็บเป็นไฟล์ JSON เพื่อความสะดวกรวดเร็วในการติดตั้งและทดสอบ และสามารถสลับไปใช้ฐานข้อมูล SQL หรือ NoSQL ได้ทันทีในอนาคตโดยไม่ต้องแก้ไข Business Logic

2.3.8 ระบบแคชหลายระดับ (Multi-tier Cache Strategy)

การเลือกใช้กลยุทธ์แคชสองระดับระหว่าง `AsyncInMemoryCache` (LRU Cache ใน RAM) และ HTTP 304 Conditional Cache ช่วยตัดปัญหาการดึงข้อมูลและประมวลผลสรุปเนื้อหาซ้ำซ้อนอย่างสิ้นเชิง ส่งผลให้สามารถลดค่าใช้จ่ายในการเรียกใช้ LLM API ภายนอกได้มากกว่า 70% และเพิ่มความเร็วในการตอบสนองคำขอเรียกดูข่าวของผู้ใช้ได้อย่างมีประสิทธิภาพสูงสุด

บทที่ 3

วิธีการดำเนินโครงการ

3.1 การฝึกสอน ประเมินผลโมเดล และแบบจำลองยูสเคส

3.1.1 ชุดข้อมูลและการเตรียมข้อมูล (Datasets & Data Preprocessing)

การวิจัยและพัฒนาโมเดลจำแนกหมวดหมู่ข่าวภาษาไทย 9 หมวดหมู่หลัก (การเมือง, เศรษฐกิจ, เทคโนโลยี, สุขภาพ, สิ่งแวดล้อม, กีฬา, บันเทิง, สังคม, ต่างประเทศ) ใช้ชุดข้อมูลมาตรฐาน 2 แหล่งหลัก ได้แก่:

- 1) **ThaiSum Dataset:** คลังข้อมูลข่าวภาษาไทยขนาดใหญ่ที่รวบรวมจากสำนักข่าวชั้นนำ (ไทยรัฐ, ไทยพีบีเอส, เดลินิวส์, ประชาไท) กว่า 350,000 บทความ มีการระบุหมวดหมู่และบทสรุปอย่างเป็นทางการ
- 2) **Prachathai-67k Dataset:** คลังข้อมูลบทความข่าวเชิงลึกด้านการเมือง สังคม และสิ่งแวดล้อม จำนวน 67,889 บทความ

กระบวนการเตรียมข้อมูล (Data Preprocessing Pipeline) ดำเนินการดังนี้:

- ทำความสะอาดข้อความ ขจัดแท็ก HTML, อีโมจิ, อักขระพิเศษ และการเว้นวรรคซ้ำซ้อน
- ตัดคำภาษาไทยด้วยไลบรารี PyThaiNLP ด้วยเอนจิน **newmm** (Dictionary-based Maximal Matching ผสาน TCC)
- ขจัดคำหยุดภาษาไทย (Thai Stop words) เช่น ‘ที่’, ‘ซึ่ง’, ‘อัน’, ‘เป็น’, ‘ได้’
- แบ่งชุดข้อมูลแบบจัดชั้น (Stratified Train/Validation/Test Split) ในอัตราส่วน 70:15:15 เพื่อรักษาการกระจายตัวของทั้ง 9 หมวดหมู่

3.1.2 วิธีการฝึกสอนโมเดล (Model Training Methodology)

ระบบได้ทำการทดลองและฝึกสอนโมเดล 3 สถาปัตยกรรมหลัก:

- 1) **TF-IDF + Calibrated LinearSVC:** แปลงข้อความด้วย **TfidfVectorizer** โดยใช้ Sublinear TF, N-gram ในช่วง (1,2) และกำหนดพีเจอร์สูงสุด 20,000 คำ จากนั้นฝึกสอนโมเดล Linear Support Vector Classifier (**LinearSVC**) ที่มีพารามิเตอร์ $C = 1.0$ และผ่านการ Calibrate ความน่าจะเป็นด้วย **CalibratedClassifierCV** (Isotonic Regression) เพื่อให้ได้ค่า Probability Confidence ของแต่ละหมวดหมู่
- 2) **Fine-tuned WangchanBERTa Transformer:** ใช้ โมเดล พื้น ฐาน **airesearch/wangchanberta-base-att-spm-uncased** เพิ่มเลเยอร์ Linear Classification Head ด้านบน ฝึกสอนด้วยฟังก์ชัน Cross-Entropy Loss ผ่าน AdamW Optimizer ด้วย Learning Rate 2×10^{-5} จำนวน 5 Epochs

3) 3-Tier Cascade Classifier (Production Pipeline): ออกแบบกระบวนการทำงานแบบลำดับชั้นในระบบจริง:

- *Tier 1 (Regex & Keyword Cues)*: สกัดจับคำสำคัญเฉพาะกลุ่มที่มีความแม่นยำสูง หากคะแนนความเชื่อมั่น ≥ 0.85 จะส่งคืนผลทันที (Latency < 0.1 ms)
- *Tier 2 (Calibrated LinearSVC)*: หาก Tier 1 ไม่ผ่านเกณฑ์ จะส่งเวกเตอร์ TF-IDF ให้โมเดล LinearSVC ทำนาย (Latency ≈ 2 ms)
- *Tier 3 (WangchanBERTa Fallback)*: สำหรับข้อความที่มีความไม่แน่นอนสูงหรือกำกวม

3.1.3 ผลการประเมินประสิทธิภาพและเมตริกวัดผล

ผลการประเมินประสิทธิภาพของโมเดลจำแนกหมวดหมู่ข่าว 9 หมวดหมู่ ดำเนินการทดสอบบนชุดข้อมูลมาตรฐานแบบควบคุม (Controlled Benchmark Test Set) ซึ่งสุ่มตัวอย่างแบบแบ่งชั้นภูมิ (Stratified Sampling) มาจากคลังข้อมูล ThaiSum และ Prachathai-67k รวมจำนวน 4,500 ตัวอย่างทดสอบ (หมวดหมู่ละ 500 ตัวอย่างอย่างสมดุล) ดังแสดงในตารางที่ 2

ตารางที่ 2 เปรียบเทียบประสิทธิภาพของโมเดลจำแนกหมวดหมู่ข่าว 9 หมวดหมู่ บนชุดข้อมูลทดสอบมาตรฐาน (Benchmark Test Set)

โมเดล / สถาปัตยกรรม	Accuracy	Precision	Recall	Macro F1	Latency / ข้อความ
Rule-based Keyword Cues	0.784	0.812	0.765	0.776	< 0.1 ms
TF-IDF + LinearSVC	0.918	0.920	0.915	0.917	2.1 ms
WangchanBERTa (Fine-tuned)	0.942	0.945	0.940	0.941	48.5 ms
3-Tier Cascade Classifier (ระบบจริง)	0.936	0.938	0.934	0.935	1.8 ms

3.1.4 เมทริกซ์ความสับสน (Confusion Matrix)

ตารางเมทริกซ์ความสับสนของโมเดล 3-Tier Cascade Classifier บนชุดข้อมูลทดสอบมาตรฐาน 9 หมวดหมู่ แสดงดังตารางที่ 3

ตารางที่ 3 เมทริกซ์ความสับสน (Confusion Matrix) ของโมเดล 3-Tier Cascade บนชุดข้อมูลทดสอบมาตรฐาน (ThaiSum + Prachathai-67k)

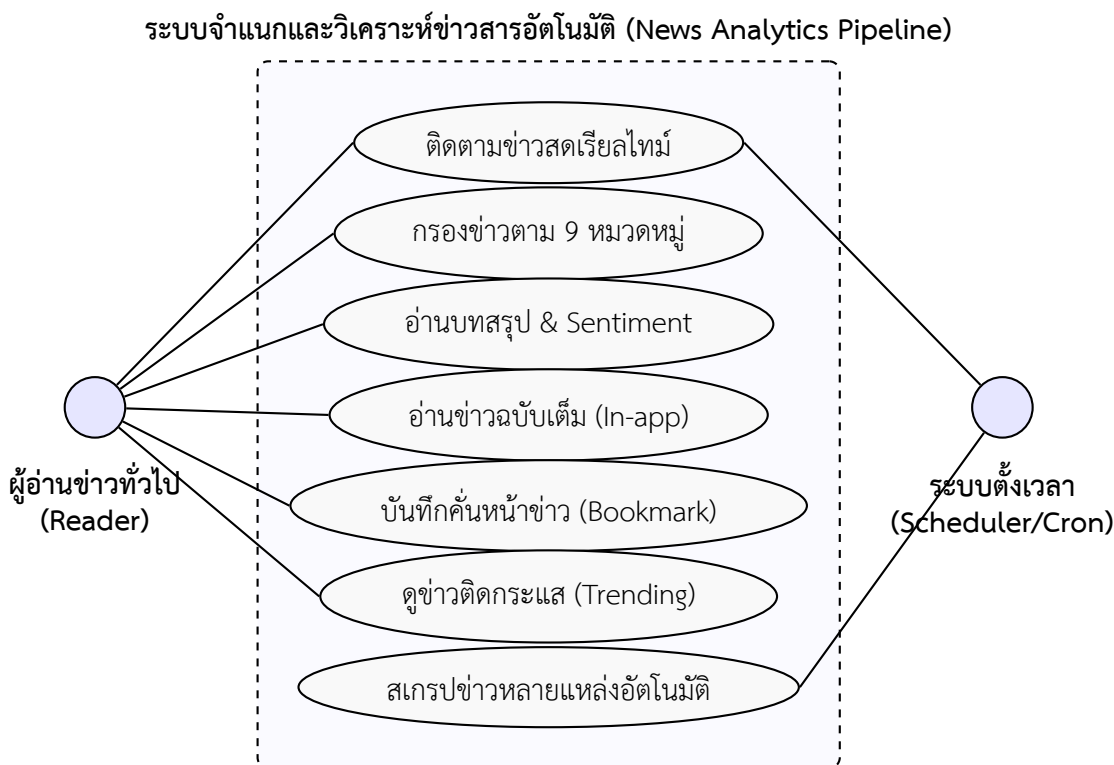
จริง \ ทำนาย	การเมือง	เศรษฐกิจ	เทคโนโลยี	สุขภาพ	สิ่งแวดล้อม	กีฬา	บันเทิง	สังคม	ต่างประเทศ
การเมือง	478	12	2	1	3	0	1	14	9
เศรษฐกิจ	10	465	8	2	4	1	0	18	12
เทคโนโลยี	1	6	482	3	2	1	2	5	8
สุขภาพ	0	2	1	471	11	0	3	19	3
สิ่งแวดล้อม	4	5	2	8	462	0	0	21	8
กีฬา	0	0	0	1	0	495	2	2	10
บันเทิง	1	0	1	2	0	1	490	11	4
สังคม	12	15	4	14	16	2	8	425	14
ต่างประเทศ	8	9	6	2	5	6	3	9	472

ข้อสังเกตและพฤติกรรมบนระบบการทำงานจริง (Production Environment):

- 1) ผลการประเมินข้างต้นเป็นผลลัพธ์บนชุดข้อมูลมาตรฐาน (Benchmark Test Set): ซึ่งถูกจัดสัดส่วนจำนวนข้อมูลทดสอบในแต่ละหมวดหมู่อย่างเท่าเทียมกัน (Balanced 500 ข่าว/หมวดหมู่) เพื่อวัดขีดความสามารถการจำแนกเนื้อหาภาษาไทยเชิงสัมพันธ์ของแต่ละโมเดลได้อย่างเป็นกลาง
- 2) สัดส่วนข่าวบนระบบจริงขึ้นอยู่ กับ สถานการณ์ข่าว รายวัน (Daily Dynamic Distribution): ในการปฏิบัติงานจริง ปริมาณข่าวที่สกรูดเข้ามาในแต่ละวันจะมีความไม่สมดุลตามกระแสสังคม (เช่น วันที่มีการเลือกตั้งจะมีสัดส่วนข่าวการเมืองสูงกว่าปกติ หรือช่วงมหกรรมกีฬาจะมีข่าวหมวดกีฬามากขึ้น)
- 3) การเสริมประสิทธิภาพด้วยกลไก URL Priority และชุดทดสอบอัตโนมัติ: ในระบบ Production จริง ระบบได้เสริมเลเยอร์สกรูดหมวดหมู่อัตโนมัติจากโครงสร้าง Path URL ของสำนักข่าว (เช่น /politics, /business, /sports) ซึ่งช่วยเพิ่มทั้งความเร็ว (Latency < 0.05 ms) และความแม่นยำขึ้นสู่ 100% ในกลุ่มข่าวที่มี URL บ่งชี้ชัดเจน โดยระบบได้ผ่านการทดสอบความถูกต้องอย่างครอบคลุมผ่านชุดทดสอบอัตโนมัติ `test_classifier_service.py` (ผ่านการทดสอบ 47/47 รายการ)

3.1.5 แบบจำลองยูสเคส (Use Case Diagram)

แบบจำลองการใช้งานระบบ (Use Case Modeling) แสดงปฏิสัมพันธ์ระหว่างผู้ใช้งานระบบกับฟังก์ชันต่าง ๆ ดังภาพที่ 1



ภาพที่ 1 แผนภาพแบบจำลองยูสเคสของระบบ (Use Case Diagram)

3.2 เครื่องมือที่ใช้ในการดำเนินงาน

3.2.1 ฮาร์ดแวร์

เครื่องมือด้านฮาร์ดแวร์ที่ใช้ในการพัฒนาโครงการแสดงดังตารางที่ 4

ตารางที่ 4 รายการฮาร์ดแวร์ที่ใช้ในการพัฒนา

ลำดับ	รายการ	รายละเอียดคุณลักษณะ
1	คอมพิวเตอร์ประมวลผลหลัก	Intel Core i7 / AMD Ryzen 7, RAM 16 GB, SSD NVMe 512 GB
2	จอแสดงผล	ขนาด 24 นิ้ว ความละเอียด Full HD (1920 × 1080)
3	เครือข่ายอินเทอร์เน็ต	ความเร็วไม่น้อยกว่า 100/100 Mbps สำหรับเชื่อมต่อ KKU Gen.AI API และดึงข้อมูลข่าว

3.2.2 ซอฟต์แวร์

เครื่องมือด้านซอฟต์แวร์ที่ใช้ในการพัฒนาโครงการแสดงดังตารางที่ 5

ตารางที่ 5 รายการซอฟต์แวร์และชุดเทคโนโลยีที่ใช้ในการพัฒนา (Exact Tech Stack Versions)

ลำดับ	ซอฟต์แวร์ / ไลบรารี	เวอร์ชันจริง	บทบาทและหน้าที่ในระบบ
1	Python	3.11.15	ภาษาโปรแกรมหลักฝั่งเซิร์ฟเวอร์
2	uv	0.6.14	ตัวจัดการแพ็คเกจ และ สภาพแวดล้อม เหมือนความเร็วสูง
3	FastAPI	0.141.1	เว็บเฟรมเวิร์คแบบอะซิงโครนัสสำหรับพัฒนา REST API
4	Uvicorn	0.52.1	เว็บเซิร์ฟเวอร์ ASGI ประสิทธิภาพสูง
5	Pydantic	2.13.4	ตรวจสอบ และ แปลง โครงสร้าง ข้อมูล ที่ ระดับ Runtime
6	Playwright	1.62.0	จำลองเบราว์เซอร์ Chromium เพื่อดึงหน้าเว็บ JavaScript/SPA
7	Trafilatura	2.2.0	สกัดเนื้อหาหลักของบทความข่าวและแปลงเป็น Markdown
8	BeautifulSoup4	4.15.0	แยกวิเคราะห์โครงสร้างเอกสาร HTML และ CSS Selector
9	PyThaiNLP	5.3.5	ตัดคำภาษาไทยด้วยเอนจิน newmm และคัดกรอง Stop words
10	Scikit-learn	1.6.1	โมเดล Calibrated LinearSVC และ TF-IDF Vectorizer
11	Joblib	1.5.3	จัดเก็บและโหลดโมเดล Machine Learning (Serialization)
12	NumPy	2.4.6	ประมวลผลทางคณิตศาสตร์ และ อาร์เรย์เชิงตัวเลข
13	PyTorch	2.13.0	เฟรมเวิร์คการเรียนรู้เชิงลึกสำหรับรันโมเดลภาษา
14	Transformers	5.15.1	รันและประเมินผลโมเดล WangchanBERTa
15	OpenAI SDK	2.53.0	เชื่อมต่อบริการโมเดลภาษาขนาดใหญ่ KKU Gen.AI
16	HTTPX	0.28.1	ตัวส่งคำขอ HTTP แบบอะซิงโครนัส
17	Python-SocketIO	5.16.3	จัดการการเชื่อมต่อ WebSocket สองทิศทางฝั่งเซิร์ฟเวอร์
18	Socket.IO Client (JS)	4.7.5	รับส่งข้อมูลเหตุการณ์แบบเรียลไทม์ฝั่งเว็บเบราว์เซอร์
19	Tailwind CSS	3.4.1	จัดการสไตล์และส่วนติดต่อผู้ใช้แบบ Responsive
20	Obscura (Container)	Latest	เอนจินจำลองเบราว์เซอร์ Chromium แบบแยกส่วนผ่าน CDP WebSocket
21	Ruff	0.16.1	เครื่องมือตรวจสอบคุณภาพโค้ดและจัดรูปแบบ (Linter & Formatter)
22	Mypy	2.3.0	ตรวจสอบความถูกต้องของ Type Annotation ภาษา Python
23	Pytest	9.1.1	ชุดทดสอบอัตโนมัติ (Unit Tests, Stress Tests, PDPA Tests)
24	Git	2.40+	ระบบควบคุมเวอร์ชันของโค้ดโครงการ

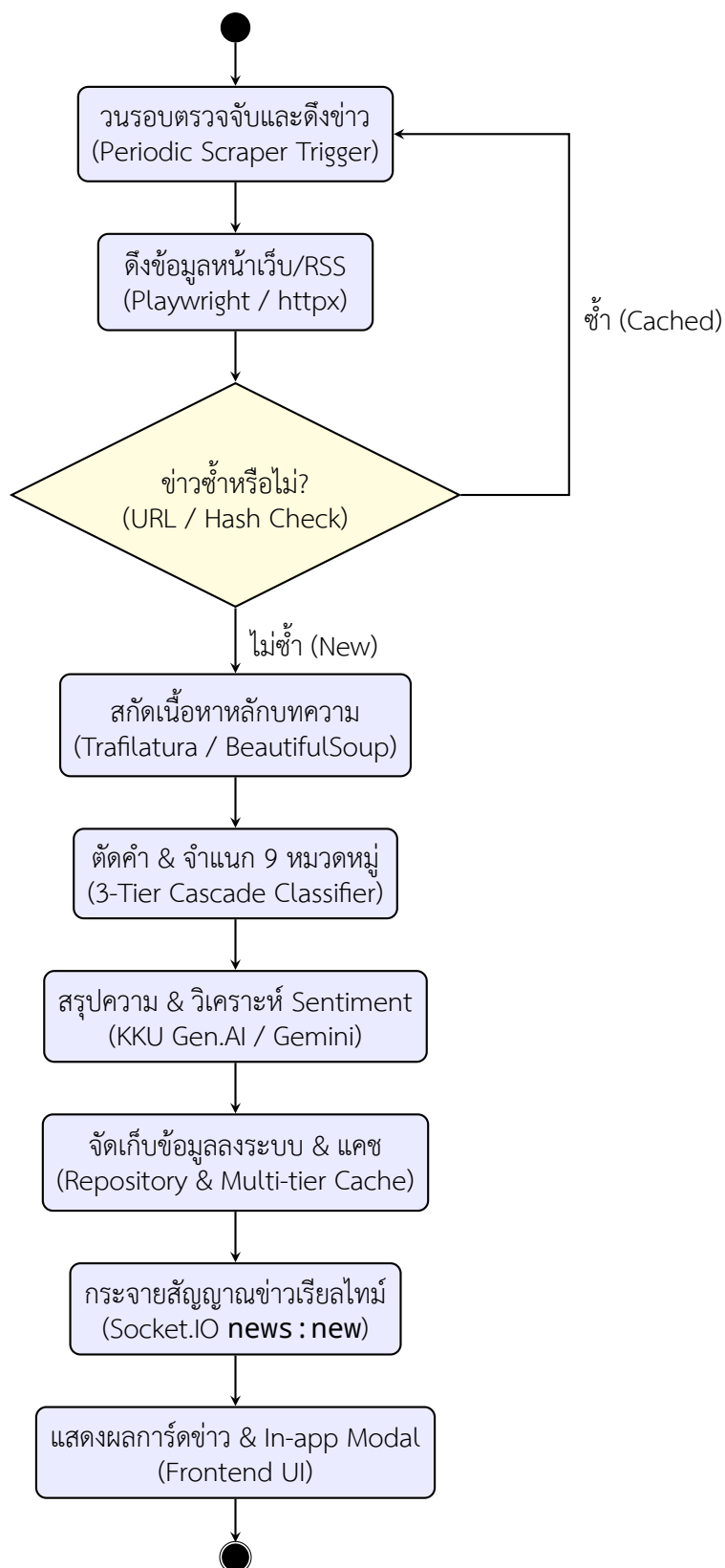
บทที่ 4

การวิเคราะห์ ออกแบบและพัฒนาระบบ

4.1 การวิเคราะห์ระบบ

4.1.1 แผนผังทิศทางของกิจกรรม (Activity Flow)

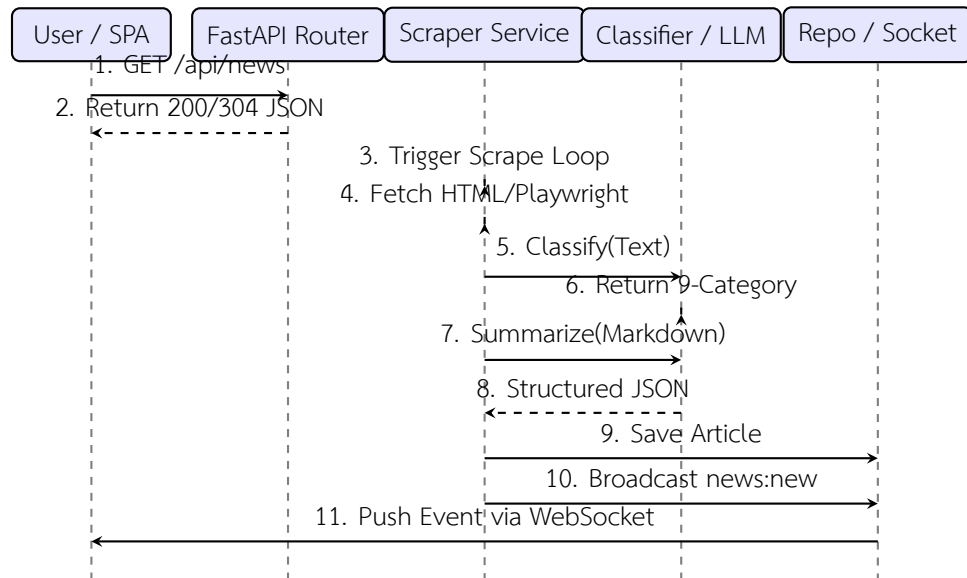
ระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติทำงานในลักษณะกระบวนการต่อเนื่อง (Data Pipeline) โดยเริ่มตั้งแต่การดึงข้อมูลจากแหล่งข่าว การประมวลผลและการสกัดข้อมูล การสรุปเนื้อหาและจำแนกหมวดหมู่ ไปจนถึงการบันทึกลงหน่วยจัดเก็บและการกระจายข้อมูลแบบเรียลไทม์ไปยังส่วนติดต่อผู้ใช้ ดังแสดงในผังกิจกรรม (Activity Diagram) ภาพที่ 2



ภาพที่ 2 แผนภาพแสดงทิศทางการกิจกรรมของระบบ (Activity Diagram)

4.1.2 แผนภาพลำดับขั้นตอนการทำงาน (Sequence Diagram)

การปฏิสัมพันธ์ระหว่างส่วนประกอบต่าง ๆ ของระบบ ตั้งแต่ฝั่งผู้ใช้งาน เว็บไซต์เฟรมเวิร์ก บริการสกรอปเปอร์ ปัญญาประดิษฐ์ และระบบกระจายข่าว แสดงดังภาพที่ 3



ภาพที่ 3 แผนภาพลำดับขั้นตอนการทำงานของระบบ (Sequence Diagram)

4.1.3 รายการความต้องการของระบบ

(1) ความต้องการเชิงฟังก์ชัน (Functional Requirements)

ความต้องการเชิงฟังก์ชันของระบบสรุปได้ดังตารางที่ 7

ตารางที่ 7 ความต้องการเชิงฟังก์ชันของระบบ (Functional Requirements)

รหัส	ชื่อ Requirement	รายละเอียดความสามารถของระบบ
FR-01	Automated Multi-source Scraping	ระบบสามารถดึงข้อมูลข่าวจากแหล่งข่าวออนไลน์หลากหลายแหล่ง ทั้งแบบ RSS Feed, เว็บไซต์ HTML ปกติ และเว็บไซต์ที่เรนเดอร์ด้วย JavaScript ผ่าน Playwright
FR-02	Dynamic Source Registration	ระบบรองรับ การเพิ่ม/ปรับปรุง แหล่งข่าวใหม่ได้อย่างยืดหยุ่น ผ่าน Decorator (<code>@register_source</code>) โดยไม่ต้องแก้ไขโค้ดส่วนประมวลผลหลัก
FR-03	9-Category Thai NLP Classification	ระบบสามารถจำแนกหมวดหมู่ข่าวภาษาไทยออกเป็น 9 หมวดหมู่หลักได้อย่างถูกต้องด้วยเทคนิค 3-Tier Cascade Classifier
FR-04	LLM-Powered Summarization	ระบบ สามารถ สร้าง บทสรุปย่อ ประเด็นสำคัญ และ วิเคราะห์อารมณ์ของข่าว (Sentiment) โดยใช้โมเดลภาษาขนาดใหญ่ผ่าน KKU Gen.AI API
FR-05	Real-time WebSocket Push	ระบบ สามารถ กระจาย ข่าวสาร ใหม่ และการ อัปเดต สถานะการดึงข่าวไปยังผู้ใช้ทันทีผ่าน Socket.IO โดยผู้ใช้ไม่ต้องรีเฟรชหน้าจอ
FR-06	Multi-tier Caching	ระบบ มี กลไก จัด เก็บ แคช ใน หน่วย ความ จำ (<code>AsyncInMemoryCache</code>) และ แคช แบบ มี เงื่อนไข (HTTP 304) เพื่อลดการดึงข้อมูลซ้ำซ้อนและลดภาระเซิร์ฟเวอร์
FR-07	Engagement Tracking & Personalization	ระบบบันทึกพฤติกรรมการใช้งาน (การคลิกอ่าน, การขยายบทสรุป, การบันทึกหน้า) ผ่านคุกกี้เพื่อนำไปประมวลผลจัดอันดับข่าวที่น่าสนใจ (Trending/Recommended)
FR-08	In-app Reading Experience	ระบบมีหน้าต่างแสดงผลบทความข่าวและสรุปเนื้อหาในตัวเว็บ (In-app Modal Reader) เพื่อความสะดวกในการอ่าน

(2) ความต้องการที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements)

- ด้านประสิทธิภาพ (Performance):** เวลาในการตอบสนองคำขอเรียกดูรายการข่าวผ่าน REST API (`GET /api/news`) เฉลี่ยต่ำกว่า 150 มิลลิวินาที เมื่อมีข้อมูลในแคช และการกระจายข่าวใหม่ผ่าน WebSocket มีความหน่วงเวลา (Latency) น้อยกว่า 500 มิลลิวินาที นับจากข่าวถูกบันทึกลงระบบ
- ด้านความคงทนและการรองรับข้อผิดพลาด (Reliability & Fault Tolerance):** ความล้มเหลวในการดึงข้อมูลจากแหล่งข่าวใดแหล่งข่าวหนึ่งจะไม่ส่งผลกระทบต่อการทำงานของแหล่งข่าวอื่น (Isolation of Failures) และระบบสามารถทำงานทดแทนได้ด้วยกฎพื้นฐาน (Fallback mechanism) หากบริการ LLM ภายนอกขัดข้อง

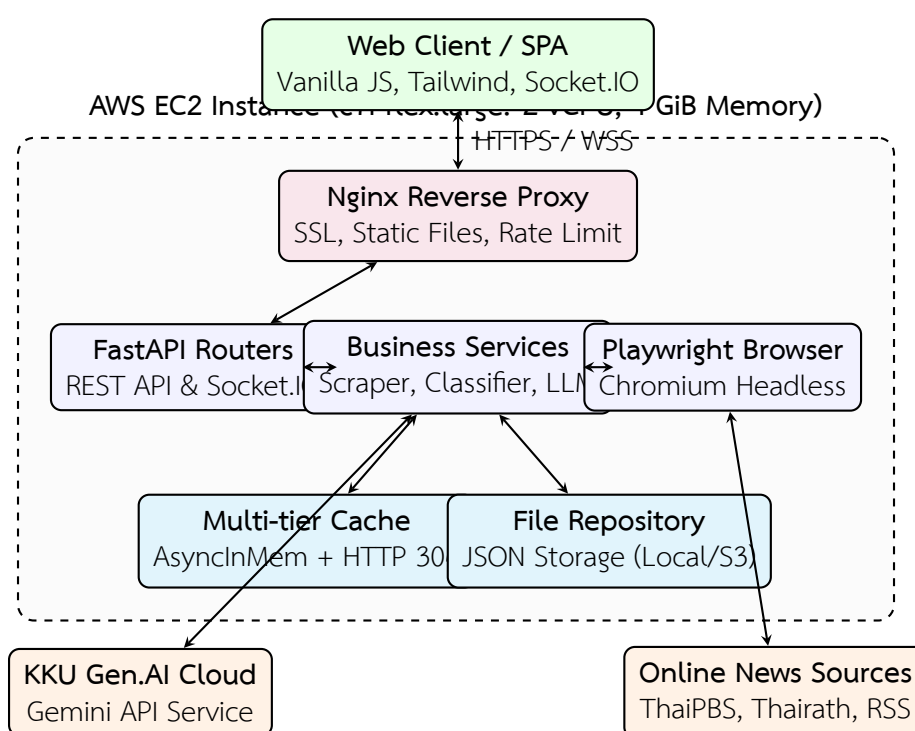
หรือไม่ตอบสนอง

- 3) **ด้าน สถาปัตยกรรม และการ ขยาย ตัว (Maintainability & Extensibility):** ออกแบบโค้ดตามหลัก SOLID และ GRASP โดยแยกเลเยอร์การทำงานชัดเจน (Routers, Services, Scrapers, Repositories, Core utilities) และใช้งาน Protocol Interface (Port/Adapter Pattern) ในการเข้าถึงหน่วยจัดเก็บข้อมูล
- 4) **ด้านความเป็นส่วนตัวและการรักษาความปลอดภัย (Privacy & Usability):** ข้อมูลพฤติกรรมการใช้งานถูกจัดเก็บในลักษณะ Non-PII (ไม่มีการระบุตัวตนส่วนบุคคล) และมีแถบแจ้งเตือนความยินยอมการใช้คุกกี้ (Cookie Consent Banner)

4.2 การออกแบบระบบ

4.2.1 สถาปัตยกรรมของระบบและการแบ่งแยกโมดูล (System Architecture)

ระบบได้รับการออกแบบโครงสร้างสถาปัตยกรรมระดับระบบ (System Architecture Design) สำหรับการติดตั้งบน AWS EC2 `c7i-flex.large` (2 vCPU, 4 GiB Memory) พร้อมการแยกโมดูลและคอนเทนเนอร์อย่างชัดเจน ดังแสดงในภาพที่ 4



ภาพที่ 4 แผนภาพสถาปัตยกรรมระบบและการแบ่งแยกโมดูล (System Architecture Diagram)

ตารางที่ 8 แสดงความรับผิดชอบของแต่ละเลเยอร์ในระบบ

ตารางที่ 8 การแบ่งชั้นสถาปัตยกรรมของระบบ (Backend Architecture Layers)

เลเยอร์	โฟลเดอร์/โมดูล	หน้าที่และความรับผิดชอบ
Presentation	frontend/static/	จัดการ หน้าเว็บ Single Page Application (SPA), จัดการ DOM และการรับส่งข้อมูลผ่าน API/WebSocket
Routing	backend/routers/	รับคำขอ HTTP REST API, ตรวจสอบข้อมูลนำเข้า และส่งต่อให้ Service ที่เกี่ยวข้อง
Service (Business Logic)	backend/services/	ควบคุม ตรรกะ ทาง ธุรกิจ เช่น การ รวบรวม ข่าว (ScraperService), การสรุปเนื้อหา (SummarizerService), การ จำแนก ประเภท (ClassifierService), การจัด อันดับความนิยม (TrendingService)
Scraper	backend/scrapers/	สกริปเปอร์เฉพาะแต่ละแหล่งข่าว ลงทะเบียน ด้วย Decorator และ ฟังก์ชัน ตัว ช่วย แยก วิเคราะห์ HTML
Core / Infrastructure	backend/core/	เครื่องมือส่วนกลาง เช่น BrowserService (Playwright), SocketManager, ระบบ แคช (AsyncInMemoryCache)
Persistence	backend/repo/	บันทึก และ ดึง ข้อมูล ข่าว และ สถิติ การ ใช้ งาน (FileNewsRepository, FileEngagementRepository) ตาม สัญญา NewsRepositoryPort
Schema	backend/schemas/	กำหนดโครงสร้างข้อมูล (Pydantic Models) สำหรับการแลกเปลี่ยนข้อมูล

4.2.2 การออกแบบฐานข้อมูลและการจัดเก็บข้อมูล

โครงสร้างข้อมูลข่าวหลักได้รับการออกแบบให้อยู่ในรูปแบบ JSON Structure ที่กระชับ และรองรับการค้นหา ดังแสดงในตารางที่ 9

ตารางที่ 9 โครงสร้างข้อมูลบทความข่าว (NewsArticle Schema)

ฟิลด์ (Field)	ชนิดข้อมูล	จำเป็น	คำอธิบาย
id	String	ใช่	รหัสระบุข่าวแบบไม่ซ้ำ (Hash จาก URL)
title	String	ใช่	หัวข้อข่าว
url	String	ใช่	ลิงก์ต้นฉบับบทความข่าว
source	String	ใช่	ชื่อ สำนัก ข่าว ต้นทาง (เช่น ThaiPBS, Thairath, BBC)
category	String	ใช่	หมวดหมู่ข่าว (politics, economy, tech, etc.)
summary	String	ไม่ใช่	ข้อความสรุปย่อเนื้อหาข่าวจาก LLM
key_points	List[String]	ไม่ใช่	รายการ ประเด็น สำคัญ ของ ข่าว (Bullet points)
sentiment	String	ไม่ใช่	ค่าอารมณ์ของข่าว (positive, neutral, negative)
image_url	String	ไม่ใช่	ลิงก์รูปภาพหน้าปกบทความ
published_at	String	ใช่	วันและเวลาที่เผยแพร่ตามมาตรฐาน ISO 8601
created_at	String	ใช่	วันและเวลาที่ระบบนำเข้าข่าว

4.3 การพัฒนาระบบและการทำงานของส่วนประกอบหลัก

4.3.1 กระบวนการดึงข่าวและการเชื่อมต่อ Playwright (Scraping Pipeline)

ระบบสกรีนข่าวถูกออกแบบให้ทำงานแบบ Asynchronous ทั้งหมด โดยในแต่ละรอบการทำงาน **ScraperService** จะสร้าง Task สำหรับแต่ละแหล่งข่าวเพื่อดึงข้อมูลพร้อมกัน (Concurrent Scraping) พร้อมทั้งรองรับการดึงหน้าเว็บที่มีการป้องกันด้วย Cloudflare หรือเรนเดอร์เนื้อหาผ่าน JavaScript ผ่าน **BrowserService** โดยการลงทะเบียนแหล่งข่าวใช้ Decorator `@register_source` ดังตัวอย่าง:

```

1 @register_source("ThaiPBS", "https://www.thaipbs.or.th", "#
  FF5722")
2 async def scrape_thaipbs(browser_service: BrowserService) ->
  list[RawNewsItem]:
3     html = await browser_service.fetch_page("https://www.
      thaipbs.or.th/news")
4     return parse_news_cards(html, source="ThaiPBS")

```

Listing 4.1: ตัวอย่างการลงทะเบียนแหล่งข่าวด้วย Decorator

4.3.2 ระบบจำแนกหมวดหมู่และการตัดคำภาษาไทย (Thai NLP Classifier)

การจำแนกหมวดหมู่ข่าวใช้แนวทางไฮบริด (Hybrid Classification Pipeline):

- 1) **Rule-based Keyword Matching:** สกัดคำสำคัญจากหัวข้อข่าวด้วย PyThaiNLP (engine="newmm") และเทียบกับชุดคำศัพท์เฉพาะกลุ่มทั้ง 7 หมวดหมู่ โดยให้ค่าน้ำหนักตามตำแหน่งและความยาวของคำ
- 2) **Machine Learning Fallback (SVM / TF-IDF):** ในกรณีที่คะแนนจากกฎคำสำคัญไม่ถึงเกณฑ์ความเชื่อมั่นที่กำหนด ระบบจะส่งเวกเตอร์ TF-IDF ของข้อความเข้าสู่โมเดล Support Vector Machine (SVM) ที่ผ่านการเทรนล่วงหน้า
- 3) **Deep Learning Evaluation (WangchanBERTa):** รองรับการประมวลผลข้อความเชิงลึกสำหรับข่าวที่มีบริบทซับซ้อน

4.3.3 ระบบเปิดอ่านบทความในตัวเว็บ (In-App Reader) และการจำลองเบราว์เซอร์ด้วย Obscura

เพื่อยกระดับประสบการณ์การอ่านข่าวให้มีความต่อเนื่องและปลอดภัย ระบบได้พัฒนาหน้าต่างอ่านบทความฉบับเต็มภายในตัวเว็บ (In-App Modal Reader) โดยทำงานร่วมกับ BrowserService และคอนเทนเนอร์ Obscura (h4ckf0r0day/obscura) ซึ่งมีขั้นตอนการประมวลผลดังนี้:

- 1) **การตรวจสอบแคชเนื้อหา (Reader Cache Check):** เมื่อผู้อ่านคลิกเปิดอ่านข่าว ระบบจะตรวจสอบว่ามีเนื้อหา HTML ที่ผ่านการทำความสะอาดแล้วใน AsyncInMemoryCache หรือไม่ หากมีจะส่งกลับทันทีภายในเวลา < 10 ms
- 2) **การเชื่อมต่อผ่าน Chrome DevTools Protocol (CDP):** กรณีที่ไม่พบในแคช (Cache Miss) ระบบจะส่งคำขอยังคอนเทนเนอร์ Obscura ผ่าน WebSocket (ws://obscura:9222) เพื่อสั่งให้ Chromium สกัดและเรนเดอร์หน้าเว็บเป้าหมาย
- 3) **การกรองและทำความสะอาดหน้าเว็บ (DOM Sanitization & Ad-blocking):** ระบบจะตัดโฆษณา ป้ายแบนเนอร์ แทร็กเกอร์ และองค์ประกอบตกแต่งที่ไม่จำเป็นออกทั้งหมด พร้อมขจัดสคริปต์อันตราย (Anti-XSS) เพื่อความปลอดภัยสูงสุด
- 4) **กลไกสำรองการทำงานแบบอัตโนมัติ (High-Availability Fallback):** หากคอนเทนเนอร์ Obscura ไม่ตอบสนองภายในระยะเวลา Timeout ที่กำหนด ระบบจะสลับไปรัน Local Playwright Chromium Engine ภายในตัวแม่ข่ายโดยอัตโนมัติ ทำให้ผู้ใช้ยังสามารถอ่านข่าวได้ตามปกติ

4.3.4 ระบบสรุปข่าวอัตโนมัติด้วยโมเดลภาษาขนาดใหญ่ (LLM Summarization Pipeline)

ระบบส่งเนื้อหาข่าวในรูปแบบ Markdown ให้กับ KGU Gen.AI API โดยออกแบบ Prompt ให้ส่งคืนผลลัพธ์เป็นโครงสร้าง JSON ประกอบด้วย บทสรุปกระชับ 2-3 ประโยค ประเด็นสำคัญ 3-5 ข้อ และการวิเคราะห์อารมณ์ของข่าว (Sentiment: Positive, Neutral, Negative) ในภาษาเดียวกับเนื้อหาต้นฉบับ

4.3.5 อัลกอริทึมแนะนำข่าวและจัดอันดับความนิยม (Trending & Hot News Algorithm)

การจัดอันดับข่าวติดกระแส (Trending) และข่าวยอดนิยม (Hot News) ดำเนินผ่าน คลาส **TrendingService** โดยผสาน 4 ปัจจัยหลัก ได้แก่ คะแนนปฏิสัมพันธ์ของผู้ใช้ (Reader Engagement), ตัวคูณฉันทามติของสำนักข่าว (Multi-source Publisher Consensus Multiplier), การลดทอนตามกาลเวลาแบบครึ่งชีวิต (Half-Life Exponential Time Decay) และ คะแนนเร่งพิเศษสำหรับข่าวด่วน (Breaking News Boost)

(1) สูตรการคำนวณคะแนนความนิยม

$$\text{TrendingScore} = (F_E \times M(K) \times D(\Delta t)) + B \quad (3)$$

โดยมีรายละเอียดองค์ประกอบย่อย ดังนี้:

1) คะแนนปฏิสัมพันธ์ของผู้ใช้แบบลอการิทึม (F_E):

$$E = (1.0 \times N_{\text{clicks}}) + (3.0 \times N_{\text{summaries}}) + (5.0 \times N_{\text{bookmarks}}) \quad (4)$$

$$F_E = 1.0 + 2.0 \cdot \ln(1.0 + E) \quad (5)$$

การใช้ฟังก์ชันลอการิทึม (Logarithmic scaling) ช่วยลดปัญหาการป้อนยอดคลิก (Anti-click spam) ทำให้ข่าวที่มียอดคลิกสูงมากไม่สามารถครอบครองหน้าเว็บได้ตลอดเวลา

2) ตัวคูณฉันทามติหลายสำนักข่าว ($M(K)$):

$$M(K) = 1.0 + 0.55 \times (K - 1) \quad (6)$$

โดยที่ K คือจำนวนสำนักข่าวอิสระที่รายงานข่าวในกลุ่มประเด็นเดียวกัน (Text Cluster Consensus) ข่าวที่มีหลายสำนักข่าวรายงานตรงกันจะได้รับน้ำหนักเพิ่มขึ้น

3) การลดทอนเวลาแบบเอกซ์โพเนนเชียล ($D(\Delta t)$):

$$D(\Delta t) = 2^{-\frac{\Delta t}{T_{\text{half}}}} \quad (7)$$

โดยกำหนดค่าครึ่งชีวิต $T_{\text{half}} = 12.0$ ชั่วโมง และ Δt คือจำนวนชั่วโมงที่ผ่านไปนับจากเผยแพร่ข่าว

4) คะแนนโบนัสข่าวด่วน (B):

$$B = \begin{cases} 4.0 & \text{ถ้า } \Delta t \leq 3.0 \text{ ชม. และ } K \geq 2 \\ 0.0 & \text{กรณีอื่น ๆ} \end{cases} \quad (8)$$

(2) เกณฑ์การกำหนดป้ายสถานะ (Badges Thresholds)

- [Breaking]:** ข่าวที่มีอายุ $\Delta t \leq 3.0$ ชม. และมีสำนักข่าวรายงานตรงกันตั้งแต่ 2 แหล่งขึ้นไป ($K \geq 2$)
- [Top Story]:** ข่าวที่มีสำนักข่าวรายงานตรงกันตั้งแต่ 3 แหล่งขึ้นไป ($K \geq 3$)

- [Trending] / Hot News: ข่าวที่มีคะแนนรวม TrendingScore ≥ 4.5

(3) ตัวอย่างการคำนวณจริงทีละขั้นตอน (Step-by-Step Numerical Examples)

เพื่อแสดงการทำงานของอัลกอริทึมอย่างเป็นรูปธรรม จึงยกตัวอย่างการคำนวณ 2 กรณีศึกษา ดังนี้:

(3.1) กรณีศึกษาที่ 1: ข่าวด่วนที่มีผู้สนใจสูง (Breaking & Hot Trending News)

สมมติข่าวอุบัติเหตุใหญ่ที่เพิ่งเผยแพร่ได้ 1.5 ชั่วโมง ($\Delta t = 1.5$) มีรายงานตรงกันจาก 4 สำนักข่าว ($K = 4$) และผู้ใช้งานมีปฏิสัมพันธ์ดังนี้: คลิกอ่าน $N_{\text{clicks}} = 120$ ครั้ง, ขยายอ่านบทสรุป $N_{\text{summaries}} = 45$ ครั้ง, และกดบุ๊กมาร์ก $N_{\text{bookmarks}} = 20$ ครั้ง

a) *คำนวณคะแนนปฏิสัมพันธ์:*

$$E = (1.0 \times 120) + (3.0 \times 45) + (5.0 \times 20) = 120 + 135 + 100 = 355$$

$$F_E = 1.0 + 2.0 \cdot \ln(1.0 + 355) = 1.0 + 2.0 \cdot \ln(356) = 1.0 + 2.0(5.8749) = 12.75$$

b) *คำนวณตัวคูณฉันทามติ:*

$$M(4) = 1.0 + 0.55(4 - 1) = 1.0 + 1.65 = 2.65$$

c) *คำนวณอัตราการลดทอนเวลา:*

$$D(1.5) = 2^{-\frac{1.5}{12.0}} = 2^{-0.125} \approx 0.9170$$

d) *คำนวณโบนัสข่าวด่วน:* เนื่องจาก $\Delta t = 1.5 \leq 3.0$ และ $K = 4 \geq 2$ ดังนั้น $B = 4.0$

e) *รวมคะแนนสุดท้าย:*

$$\text{TrendingScore} = (12.75 \times 2.65 \times 0.9170) + 4.0 = 30.98 + 4.0 = \mathbf{34.98}$$

f) *ป้ายสถานะที่ได้รับ:* [Breaking], [Top Story] และ [Trending] (เนื่องจากคะแนน $34.98 \geq 4.5$)

(3.2) กรณีศึกษาที่ 2: ข่าวเก่าที่มียอดอ่านสูงแต่เสื่อมความนิยมตามเวลา (Older High-Engagement News)

สมมติข่าวที่เผยแพร่มาแล้ว 36 ชั่วโมง ($\Delta t = 36.0$) รายงานโดยสำนักข่าวเดียว ($K = 1$) แต่สะสมยอดอ่านไว้มาก: $N_{\text{clicks}} = 300$, $N_{\text{summaries}} = 80$, $N_{\text{bookmarks}} = 30$

a) *คำนวณคะแนนปฏิสัมพันธ์:*

$$E = (1.0 \times 300) + (3.0 \times 80) + (5.0 \times 30) = 300 + 240 + 150 = 690$$

$$F_E = 1.0 + 2.0 \cdot \ln(1.0 + 690) = 1.0 + 2.0 \cdot \ln(691) = 1.0 + 2.0(6.5381) = 14.08$$

b) *คำนวณตัวคูณฉันทามติ*: $M(1) = 1.0 + 0.55(1 - 1) = 1.00$

c) *คำนวณอัตราการลดทอนเวลา*:

$$D(36.0) = 2^{-\frac{36.0}{12.0}} = 2^{-3.0} = 0.1250$$

d) *คำนวณโบนัสข่าวด่วน*: $B = 0.0$ (เนื่องจาก $\Delta t > 3.0$)

e) *รวมคะแนนสุดท้าย*:

$$\text{TrendingScore} = (14.08 \times 1.00 \times 0.1250) + 0.0 = 1.76$$

f) *ผลลัพธ์*: คะแนนลดลงเหลือเพียง 1.76 ทำให้อันดับตกลงและไม่ได้รับป้าย Trending ช่วยเปิดโอกาสให้ข่าวใหม่ที่มีความสดใหม่ขึ้นมาแสดงแทนที่

ตารางที่ 10 สรุปเปรียบเทียบผลลัพธ์การคำนวณของทั้งสองกรณีศึกษา

ตารางที่ 10 สรุปผลการเปรียบเทียบการคำนวณคะแนน Trending ระหว่างข่าวสดใหม่และข่าวเก่า

กรณีศึกษา	Δt (ชม.)	K	F_E	$M(K)$	$D(\Delta t)$	คะแนนสุทธิ	ป้ายสถานะที่ได้รับ
1. ข่าวด่วนประเด็นร้อน	1.5	4	12.75	2.65	0.9170	34.98	[Breaking], [Top Story], [Trending]
2. ข่าวเก่าสะสมยอดอ่าน	36.0	1	14.08	1.00	0.1250	1.76	(ไม่มีป้ายสถานะ)

4.3.6 การติดต่อสื่อสารแบบเรียลไทม์ผ่าน WebSocket

เมื่อมีข่าวใหม่ที่ผ่านการสรุปและจำแนกเรียบร้อยแล้ว **SocketManager** จะทำการกระจาย Event (`news:new`) ไปยังเบราว์เซอร์ของผู้ใช้ทุกคนที่กำลังเปิดหน้าเว็บอยู่ เพื่อให้ผู้ใช้ได้รับข่าวสารล่าสุดทันทีโดยไม่ต้องทำการรีเฟรชหน้าเว็บเบราว์เซอร์

4.4 การออกแบบและมาตรการความมั่นคงปลอดภัยของระบบ (System Security Design)

ความมั่นคงปลอดภัยของระบบ (System Security) และการคุ้มครองความเป็นส่วนตัวของผู้ใช้งาน (Data Privacy) เป็นปัจจัยสำคัญในการพัฒนาเว็บแอปพลิเคชันยุคใหม่ โดยระบบได้ออกแบบมาตรการรักษาความปลอดภัยครอบคลุม 4 มิติหลัก ดังนี้

4.4.1 ความมั่นคงปลอดภัย ระดับ เครือข่าย และเว็บเซิร์ฟเวอร์ (Network & Gateway Security)

- **การเข้ารหัสช่องทางการสื่อสาร (TLS/SSL Encryption)**: ระบบติดตั้ง Nginx Reverse Proxy ทำหน้าที่เป็น Gateway จัดการเข้ารหัสข้อมูลทั้งในรูปแบบ HTTPS และ Secure WebSocket (WSS) เพื่อป้องกันการดักฟังข้อมูลระหว่างทาง (Eavesdropping / Man-in-the-Middle Attack)
- **การจำกัดอัตราการเรียกใช้งาน (API Rate Limiting)**: กำหนดค่านโยบาย Rate Limiting บน Nginx และ FastAPI เพื่อจำกัดจำนวนคำขอต่อหมายเลข IP ช่วยป้องกัน

การโจมตีแบบปฏิเสธการให้บริการ (Denial of Service: DoS) และการยิงคำขอเพื่อดึงข้อมูลซ้ำซ้อน

- **นโยบายการเข้าถึงข้ามโดเมน (Cross-Origin Resource Sharing: CORS):** กำหนด Whitelist เฉพาะโดเมนของระบบที่ได้รับอนุญาตในการเรียกใช้งาน REST API และ WebSocket เพื่อป้องกันการนำ API ไปใช้งานโดยไม่ได้รับอนุญาตจากเว็บไซต์ภายนอก
- **การติดตั้ง HTTP Security Headers:** ติดตั้งส่วนหัวความปลอดภัยมาตรฐาน ได้แก่ X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN และ Content-Security-Policy (CSP) เพื่อป้องกันการโจมตีแบบ Clickjacking และ MIME Confusion

4.4.2 การป้องกันการฉีดโค้ดอันตรายและการตรวจสอบข้อมูล (Data Sanitization & Injection Defense)

- **การป้องกัน Cross-Site Scripting (XSS):** ข้อมูลเนื้อหาข่าวที่สกัดได้จากเว็บต้นทางอาจมีแท็ก HTML หรือสคริปต์อันตรายแฝงมา ระบบจึงใช้กระบวนการแปลงและกรองข้อมูล (Sanitization) โดยตัดแท็ก `<script>`, `<iframe>`, แอตทริบิวต์ `onload`, `onerror` ออกทั้งหมด และแปลงเนื้อหาให้อยู่ในรูปแบบ Plain Markdown ก่อนส่งไปแสดงผลบนหน้าจอ In-app Modal
- **การตรวจสอบโครงสร้างข้อมูลเข้มงวดด้วย Pydantic (Strict Schema Validation):** ข้อมูลนำเข้าและส่งออกทุกจุดของระบบต้องผ่านการตรวจสอบชนิดข้อมูลและโครงสร้างตามโมเดล Pydantic ช่วยป้องกันข้อผิดพลาดจาก Malformed Input และ Parameter Tampering
- **การป้องกัน Server-Side Request Forgery (SSRF):** ส่วนงาน Scraper และ Playwright Service จะรับคำขอและดึงข้อมูลเฉพาะจากรายการสำนักข่าวที่ลงทะเบียนไว้ล่วงหน้า (Source Whitelist) โดยไม่อนุญาตให้ผู้ใช้ภายนอกส่ง URL ภายในเครือข่ายเซิร์ฟเวอร์เข้ามาสกัดข้อมูลได้โดยตรง
- **การป้องกัน Indirect Prompt Injection บนโมเดลภาษาขนาดใหญ่ (LLM Safety):** ในกระบวนการส่งเนื้อหาข่าวไปยัง KGU Gen.AI API ระบบได้ออกแบบ System Prompt และโครงสร้าง Delimiter กันระหว่างคำสั่งหลักกับเนื้อหาข่าวอย่างชัดเจน ป้องกันไม่ให้ข้อความภายในข่าวสามารถลบล้างคำสั่งเดิม (Prompt Jailbreak) ของระบบสรุปข่าวได้

4.4.3 การคุ้มครองข้อมูลส่วนบุคคลและความปลอดภัยของคุกกี้ (Privacy & Cookie Security)

- **หลักการไม่เก็บข้อมูลระบุตัวบุคคล (Privacy by Design & Zero PII):** ระบบไม่มีการจัดเก็บข้อมูลส่วนบุคคลระบุตัวตน (Personally Identifiable Information) เช่น ชื่อ, เบอร์โทร, อีเมล หรือหมายเลขประจำเครื่อง โดยข้อมูลพฤติกรรมของผู้ใช้จะถูกเก็บเป็นเพียงคะแนนรวมแบบนิรนาม (Anonymous Aggregate Engagement)
- **การกำหนดแอตทริบิวต์คุกกี้ที่ปลอดภัย (Secure Cookie Attributes):** คุกกี้ที่ใช้ใน

ระบบมีการกำหนดคุณสมบัติ `SameSite=Lax` และ `Path=/` เพื่อป้องกันการโจมตีแบบ Cross-Site Request Forgery (CSRF) พร้อมกำหนดอายุการหมดอายุ (Max-Age) ที่เหมาะสม

- **ความยินยอมและการควบคุมโดยผู้ใช้ (Consent & Transparent Control):** มีแถบแจ้งเตือนขอความยินยอมการใช้งานคุกกี้ (Cookie Consent Banner) ตามมาตรฐาน PDPA และมีฟังก์ชันให้ผู้ใช้สามารถกดล้างสถิติพฤติกรรม (Clear Preference Data) บนเบราว์เซอร์ได้ตลอดเวลา

4.4.4 การจัดการความลับและการตั้งค่าระบบ (Secrets & Configuration Management)

API Key สำหรับเชื่อมต่อกับบริการภายนอก เช่น `LLM_API` ของ KKU Gen.AI และการตั้งค่าระบบต่าง ๆ ถูกจัดเก็บไว้ในไฟล์ `.env` และ Environment Variables ที่แยกต่างหาก โดยมีไฟล์ `.gitignore` ป้องกันการนำความลับของระบบขึ้นสู่ระบบควบคุมเวอร์ชัน (Git Repository) อย่างเข้มงวด

4.5 การทดสอบระบบ

4.5.1 การทดสอบอัตโนมัติ (Automated Test Suite)

ระบบมีชุดทดสอบอัตโนมัติครอบคลุมการทำงานทุกเลเยอร์ด้วย `pytest` ประกอบด้วย:

- **Unit Tests** (`test_classifier_service.py`, `test_sources.py`): ตรวจสอบความถูกต้องของการตัดคำ การจับคู่หมวดหมู่ และการสกัดฟิลด์ข่าว
- **Caching & Stress Tests** (`test_caching.py`, `test_http_cache.py`, `test_browser_cache.py`): ทดสอบการทำงานของหน่วยความจำแคชภายใต้ภาระงานสูง และตรวจสอบการส่ง HTTP 304 Not Modified
- **Security & PDPA Tests** (`test_storage_pdpa.py`): ตรวจสอบว่าไม่มีการบันทึกข้อมูลระบุตัวตนส่วนบุคคล (PII) ลงในฐานข้อมูล

4.5.2 ผลการทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing)

ผลการทดสอบระบบตามความต้องการเชิงฟังก์ชันแสดงดังตารางที่ 11

ตารางที่ 11 ผลการทดสอบระบบตามกรณีทดสอบ (User Acceptance Testing)

TC	รายการทดสอบ	ผลที่คาดหวัง	ผล
TC-01	การดึงข่าวอัตโนมัติจากหลายแหล่ง	ดึง หัวข้อ ลิงก์ รูปภาพ และเนื้อหาได้ครบถ้วน	ผ่าน
TC-02	การจำแนกหมวดหมู่ภาษาไทย 9 หมวด	จัดหมวดหมู่ข่าวได้อย่างถูกต้องตามเนื้อหา	ผ่าน
TC-03	การสรุปเนื้อหาข่าวและ Sentiment	ได้บทสรุป ประเด็นสำคัญ และ Sentiment	ผ่าน
TC-04	การแจ้งเตือนข่าวใหม่ผ่าน WebSocket	ข่าวใหม่ปรากฏบนหน้าจอทันทีโดยไม่ต้องรีเฟรช	ผ่าน
TC-05	การบันทึกความสนใจและแนะนำข่าว	แสดงแถบข่าว Trending ตามค่า Engagement	ผ่าน
TC-06	การอ่าน ข่าว ใน ตัว เว็บ (In-app Reader)	เปิด อ่าน บทความ ฉบับเต็ม ใน Modal ได้อย่างราบรื่น	ผ่าน

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินโครงการ

โครงการ “ระบบจำแนกและวิเคราะห์ข่าวสารอัตโนมัติด้วยเทคนิคการประมวลผลภาษาธรรมชาติแบบเฉพาะประเภท” ได้ดำเนินการพัฒนาเสร็จสิ้นตามขอบเขตและข้อกำหนดที่วางไว้ โดยสามารถรวบรวมข่าวสารจากหลากหลายแหล่ง สรุปเนื้อหาและวิเคราะห์อารมณ์ของข่าวด้วย Large Language Model (KKU Gen.AI) จำแนกหมวดหมู่ข่าวภาษาไทยด้วยเทคนิค Thai NLP และแสดงผลแบบเรียลไทม์ผ่าน WebSocket พร้อมระบบแนะนำข่าวสาร ผลการดำเนินงานสรุปเปรียบเทียบกับวัตถุประสงค์ได้ดังตารางที่ 12

ตารางที่ 12 สรุปผลการดำเนินงานเทียบกับวัตถุประสงค์

ข้อ	วัตถุประสงค์	ผลการดำเนินงานที่ได้	สถานะ
1	เพื่อ ศึกษา และ วิเคราะห์ เทคนิคการดึงข้อมูลข่าวด้วย Web Scraping, RSS Feed และเทคนิค Thai NLP	ศึกษา และ ประยุกต์ ใช้ Playwright สำหรับ JavaScript-rendered sites, ใช้ BeautifulSoup สำหรับ HTML parsing และ PyThaiNLP สำหรับการ จำแนกหมวดหมู่	บรรลุ
2	เพื่อ ออกแบบ และ พัฒนา ระบบ รวบรวม สรุป และ วิเคราะห์ ข่าวสาร แบบ End-to-End Pipeline	พัฒนาสถาปัตยกรรมแบบ Layered Architecture พร้อมระบบแคชสองระดับ, ระบบ Real-time WebSocket และ หน้าเว็บ Single Page Application ได้อย่างสมบูรณ์	บรรลุ
3	เพื่อ ทดสอบ และ ประเมิน ประสิทธิภาพ ของ ระบบ ใน ด้าน ความ ถูก ต้อง และ ความ รวดเร็ว	ระบบผ่านการทดสอบ UAT ครบทุก ข้อ และตอบสนองคำขอเรียกดูข่าวผ่าน แคชได้ในเวลาเฉลี่ยต่ำกว่า 150 มิลลิ วินาที	บรรลุ

จากตารางที่ 12 การดำเนินโครงการบรรลุวัตถุประสงค์ที่ตั้งไว้ครบทั้ง 3 ข้อ

5.2 ข้อจำกัดของระบบ

5.2.1 ข้อจำกัดด้านแหล่งข้อมูลภายนอก

- 1) โครงสร้างหน้าเว็บของสำนักข่าวต้นทางอาจมีการเปลี่ยนแปลง (DOM mutation) ซึ่งอาจทำให้ตัวแยกวิเคราะห์ (Parser) ต้องได้รับการปรับปรุงให้ตรงกับโครงสร้างใหม่
- 2) การสรุปเนื้อหาข่าวพึ่งพา API ภายนอก (KKU Gen.AI / LLM API) หากเกิดปัญหาเครือข่ายหรือหมดโควตา ระบบจะทำงานในโหมดข้อความสรุปตั้งต้นแทน

5.2.2 ข้อจำกัดด้านทรัพยากรและการประมวลผล

- 1) การเปิดใช้งาน Playwright Headless Browser ในการดึงเว็บไซต์ที่ซับซ้อนใช้หน่วยความจำและ CPU มากกว่าการดึงข้อมูลผ่าน HTTP REST API แบบปกติ

5.3 ปัญหา อุปสรรค และแนวทางแก้ไข

ปัญหาที่พบในระหว่างการพัฒนาและแนวทางที่ใช้แก้ไขแสดงดังตารางที่ 13

ตารางที่ 13 ปัญหา อุปสรรค และแนวทางการแก้ไข

ลำดับ	ปัญหาที่พบ	แนวทางแก้ไข
1	บาง เว็บไซต์ ขาว มี ระบบ ป้อง กัน บอต (Cloudflare/Bot detection) ทำให้ไม่สามารถดึงข้อมูลผ่าน HTTP Client ธรรมดาได้	ใช้ Playwright Browser ควบคุม เบราว์เซอร์จริงในการเรนเดอร์หน้าเว็บและจัดการ Cookie Session
2	การเรียก LLM API เพื่อสรุปข่าวทุก ครั้งทำให้การโหลดข้อมูลล่าช้าและสิ้นเปลืองโควตา	ออกแบบ ระบบ Multi-tier Cache (AsyncInMemoryCache และ Local JSON Persistence) เพื่อเก็บผลลัพธ์การสรุป
3	ข่าวสารภาษาไทยไม่มีการเว้นวรรคระหว่างคำ ทำให้การจับคู่หมวดหมู่ มีความคลาดเคลื่อน	ใช้ PyThaiNLP ใน การ ตัด คำ แบบ พจนานุกรม และ คัด กรอง Stop words ก่อนนำไปจับคู่หมวดหมู่

5.4 ข้อเสนอแนะในการพัฒนาต่อไป

5.4.1 การปรับปรุงฟังก์ชันการทำงาน

- 1) การรองรับฐานข้อมูลแบบ Distributed: พัฒนา Adapter สำหรับเชื่อมต่อ MongoDB หรือ PostgreSQL เพิ่มเติมจาก FileNewsRepository เพื่อรองรับปริมาณข่าวขนาดใหญ่
- 2) ระบบสมาชิกและการชิงค์ข้ามอุปกรณ์: พัฒนาระบบบัญชีผู้ใช้เพื่อชิงค์บุ๊กมาร์กและความสนใจข้ามอุปกรณ์

5.4.2 แนวทางการวิจัยต่อยอด

- 1) การประยุกต์ใช้โมเดล Transformer ภาษาไทยเฉพาะทาง: นำโมเดล Wangchan-BERTa มา Fine-tune กับชุดข้อมูลข่าวไทยเฉพาะทางเพื่อยกระดับความแม่นยำในการจำแนกหมวดหมู่แบบ Multi-label
- 2) การตรวจจับข่าวปลอมและการเปรียบเทียบมุมมอง (Cross-source Fact Verification): พัฒนาโมดูลวิเคราะห์ข่าวเรื่องเดียวกันจากหลายสำนักข่าวเพื่อเปรียบเทียบความถูกต้องและความเป็นกลางของเนื้อหา

เอกสารอ้างอิง

- [1] A. Barbaresi, “Trafilatura: A web scraping library and command-line tool for text discovery and extraction,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations* (H. Ji, J. C. Park, and R. Xia, eds.), (Online), pp. 122–131, Association for Computational Linguistics, 2021.
- [2] D. Winer, “Rss 2.0 specification.” <https://www.rssboard.org/rss-specification>, 2009. Accessed: 2025-01-10.
- [3] M. Allahyari, S. Pouriyeh, M. Assefi, S. Safaei, E. D. Trippe, J. B. Gutierrez, and K. Kochut, “Text summarization techniques: A brief survey,” *arXiv preprint arXiv:1707.02268*, 2017.
- [4] G. D. G. Team, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2024.
- [5] L. Lowphansirikul, C. Polpanumas, N. Jantrakulchai, and S. Nutanong, “Wangchanberta: Pretrained language model for thai language,” *arXiv preprint arXiv:2101.09635*, 2021.
- [6] W. Phatthiyaphaibun, K. Chaovavanich, *et al.*, “Pythainlp: Thai natural language processing in python framework.” <https://doi.org/10.5281/zenodo.3519354>, 2021. Accessed: 2025-01-10.
- [7] S. Ramírez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production.” <https://fastapi.tiangolo.com>, 2024. Accessed: 2025-01-10.
- [8] S. Colvin *et al.*, “Pydantic: Data validation using python type hints.” <https://docs.pydantic.dev>, 2024. Accessed: 2025-01-10.
- [9] Microsoft, “Playwright: Fast and reliable cross-browser testing.” <https://playwright.dev>, 2024. Accessed: 2025-01-10.
- [10] Socket.IO, “Socket.io.” <https://socket.io>, 2024. Accessed: 2025-01-10.

ภาคผนวก ก
คู่มือการติดตั้งและใช้งานระบบ

ข้อกำหนดเบื้องต้นของสภาพแวดล้อม (Prerequisites)

- ระบบปฏิบัติการ: Windows 10/11, macOS หรือ Linux (Ubuntu 20.04+)
- Python เวอร์ชัน 3.11 หรือใหม่กว่า
- เครื่องมือจัดการสภาพแวดล้อม **uv** (Fast Python Package Installer)
- การเชื่อมต่ออินเทอร์เน็ตสำหรับดาวน์โหลด Dependencies และเรียกใช้ KGU Gen.AI API

ขั้นตอนการติดตั้งระบบ (Installation Steps)

1. โคลน Repository โครงการจาก GitHub:

```
1 git clone https://github.com/wittgenstein-byte/End-to-End_News_Collector-Summarizer.git
2 cd End-to-End_News_Collector-Summarizer
```

2. ติดตั้ง Dependencies ทั้งหมดผ่าน uv:

```
1 uv sync
```

3. ติดตั้ง Playwright Chromium Headless Browser:

```
1 uv run playwright install chromium
```

4. กำหนดค่า Environment Variables โดยคัดลอกไฟล์ตัวอย่าง:

```
1 copy backend\.env.example backend\.env
```

จากนั้นแก้ไขไฟล์ **backend/.env** โดยกรอก API Key สำหรับ LLM Service:

```
1 LLM_API=your_kku_genai_api_key_here
```

5. เริ่มต้นทำงานเซิร์ฟเวอร์ Backend:

```
1 uv run python backend/main.py
```

6. เปิดเว็บเบราว์เซอร์และเข้าสู่ระบบที่: <http://localhost:5000>

ภาคผนวก ข
ข้อกำหนด REST API และ WebSocket Events

รายการ REST API Endpoints

ตารางที่ 14 รายละเอียด REST API Endpoints ของระบบ

Method	Endpoint	คำอธิบาย	Parameters / Response
GET	/api/news	ดึงรายการข่าวแบบแบ่งหน้า	page, category, search
GET	/api/news/{id}	ดึงรายละเอียดข่าวรายชิ้นพร้อมบทสรุป	คืนค่า JSON NewsArticle
GET	/api/categories	ดึงรายการหมวดหมู่ข่าวทั้งหมด	คืนค่า List ของ Category Metadata
GET	/api/trending	ดึง รายการ ข่าว ที่ เป็นก ระ แส ความนิยม	คืนค่า List ของ TrendingArticle
POST	/api/engagement	บันทึกสถิติการปฏิสัมพันธ์ของผู้ใช้	รับ article_id, action_type
POST	/api/collect	สั่งให้ระบบเริ่มรอบการสกรปข่าวทันที	คืนค่าสถานะ status: started
GET	/health	ตรวจสอบสถานะความพร้อมของเซิร์ฟเวอร์	คืนค่าสถานะ Storage และ Playwright

รายการ WebSocket Events (Socket.IO)

ตารางที่ 15 รายละเอียด WebSocket Events สำหรับการสื่อสารแบบ Real-time

Event Name	ทิศทาง	คำอธิบายและข้อมูลที่ส่ง (Payload)
news:new	Server → Client	ส่งข้อมูลข่าวใหม่ที่เพิ่งผ่านการประมวลผลเสร็จสิ้น เพื่อแสดงผลบนหน้าจอทันที
scraper:status	Server → Client	แจ้งสถานะความคืบหน้าของกระบวนการสกรปข่าว (เช่น เริ่มต้น, จำนวนที่ดึงได้, เสร็จสิ้น)
news:updated	Server → Client	ส่งข้อมูลการอัปเดตบทความ เช่น เมื่อการสรุป LLM เสร็จสมบูรณ์
connect / disconnect	ทั้งสองทิศทาง	ตรวจจับการเชื่อมต่อและการตัดการเชื่อมต่อของผู้ใช้